

Food Surplus Redistribution and Community Hunger Reduction Platform

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

PAILA NAVEEN	2520030518
KATTA YAMINI SAI	2520030380
KODURU REVANTH	2520030545

Course: DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT

Group Number: 18

Under the Esteemed Guidance of

Dr Yerragudipadu Subbarayudu

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled “**Food Surplus Redistribution and Community Hunger Reduction Platform**” is a record of Bonafide work of **PAILA NAVEEN - 2520030518**, **KATTA YAMINI SAI - 2520030380**, **KODURU REVANTH - 2520030545** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University, as part of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT**. The results embodied in this report have not been copied from any other departments/University/Institute.

PAILA NAVEEN	2520030518
KATTA YAMINI SAI	2520030380
KODURU REVANTH	2520030545

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled “**Food Surplus Redistribution and Community Hunger Reduction Platform**” is a bonafide work done and submitted by **PAILA NAVEEN - 2520030518, KATTA YAMINI SAI - 2520030380, KODURU REVANTH - 2520030545** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), for the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT**, during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator **Dr Yerragudipadu Subbarayudu** and our guide **Dr Yerragudipadu Subbarayudu**, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance throughout the course “*Database Systems Engineering And Distributed Backend Development*” cannot be expressed in words.

We express our gratitude to **Dr. P Venkateshwara Rao**, Head of the *Department of Computer Science and Engineering*, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and all those who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

PAILA NAVEEN	2520030518
KATTA YAMINI SAI	2520030380
KODURU REVANTH	2520030545

ABSTRACT

Every day, large quantities of perfectly edible food from restaurants, hotels, banquet halls, supermarkets, events and households are thrown away, while many families and communities struggle to obtain even one nutritious meal. The UNEP Food Waste Index Report 2024 estimates that the world wasted about 1.05 billion tonnes of food in 2022, even though roughly 783 million people were hungry. The gap is rarely caused by a lack of food or generosity. It is caused by the lack of an organised, real-time system that connects those who have surplus food with those who can collect and distribute it before it spoils.

This project, the Food Surplus Redistribution and Community Hunger Reduction Platform, addresses that gap through a polyglot, microservice-based backend developed as part of the course Database Systems Engineering and Distributed Backend Development. Food donors publish surplus listings with food type, quantity, location, preparation time and safe-consumption deadline. NGOs and volunteers discover nearby donations, reserve the required quantity, schedule pickups and record distribution to beneficiaries. The complete journey of every meal from donor to pickup to NGO to beneficiary is tracked and auditable.

The system applies the full course outcomes. A normalised PostgreSQL schema with constraints, indexes, views, stored procedures, triggers and ACID transactions forms the relational core. MongoDB stores flexible user profiles, activity streams and notifications, and pgvector supports semantic matching and a retrieval-augmented food-safety assistant. The backend is split into an API Gateway and a Matching and Insight service built with FastAPI, a User and Activity service built with Node.js, Express and Socket.io, and a Food Listing and Pickup service built with Spring Boot. Services own their data, communicate over REST, and use the Saga pattern with compensating actions for distributed consistency. The platform is containerised with Docker, orchestrated on Kubernetes, monitored with OpenTelemetry, Prometheus and Grafana, and delivered through a GitHub Actions pipeline with image scanning and OWASP-guided security checks.

By turning surplus food into a tracked, matched and delivered resource, the platform aims to reduce food wastage, improve the utilisation of donated food and help communities that need it most.

INDEX

S. No.	Chapters	Topics	Page no.
		Acknowledgement	4,4
		Abstract	5,7
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives of the Project 1.5 Importance of the Application 1.6 Target Users / Audience 1.7 Mapping of the Project to Course Outcomes 1.8 Organisation of the Report	8 - 14
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks 2.4 Functional Requirements 2.5 Non-Functional Requirements	15 - 18
3	Technology Stack	3.1 Front-End Design 3.2 Back-End Design 3.3 Database Design 3.4 Version Control 3.5 APIs or External Services	19 - 23
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer (Frontend, Backend, Database) 4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)	24 - 30
5	Design	5.1 Design Overview and Use Cases 5.2 ER Diagram / Database Schema 5.3 Normalisation 5.4 Index Design 5.5 Views, Stored Logic and Transaction Design 5.6 MongoDB Schema Design 5.7 Vector Database Design 5.8 API Contract Design	31 - 42

S. No.	Chapters	Topics	Page no.
6	Implementation	6.1 Module-Wise Implementation 6.2 Front-End Logic (UI Rendering and State Management) 6.3 API Integration 6.4 Authentication and Authorisation 6.5 Database Programming: Advanced SQL, Views, Procedures, Triggers and Transactions 6.6 MongoDB Implementation 6.7 Vector Search and Retrieval-Augmented Assistant 6.8 Distributed Consistency: Saga, Compensation and Cross-Database Integrity 6.9 Refactoring the Monolith into Layers and Services	43 - 58
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	59 - 62
8	Testing	8.1 Postman Testing (Frontend/Backend) 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	63 - 68
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	69 - 75
10	Challenges & Limitations	10.1 Technical Challenges Faced 10.2 Team Coordination Challenges 10.3 Current Limitations of the System	76 - 78
11	Future Enhancements	11.1 Planned Features 11.2 Possible Scalability Improvements	79 - 80
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned During Development	81 - 83
13	References	13.1 Books 13.2 Research Papers and Technical Reports 13.3 Official Documentation	84 - 86

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

Food is one of the most basic human needs, yet the world's food systems waste it on an enormous scale. The United Nations Environment Programme reports that about 1.05 billion tonnes of food were wasted in 2022, which is close to one-fifth of all food available to consumers, and that households, food services and retail outlets account for roughly 60 percent, 28 percent and 12 percent of that waste respectively [16]. During the same period, hundreds of millions of people were classified as hungry. The contradiction is obvious: edible food is discarded in one part of a city while another part of the same city goes without.

In India the picture is familiar to everyone. Weddings, corporate events, hostel messes, restaurants, hotels and supermarkets routinely prepare more food than is eventually consumed. Most of this surplus is still perfectly safe and nutritious at the moment it becomes surplus. Its value, however, is short-lived. Cooked food has a safe-consumption window that is measured in hours, not days. If a donor cannot find an NGO or volunteer within that window, the food is thrown away. Many donors are willing to give, and many NGOs are willing to collect, but there is no dependable channel that tells them about each other at the right time.

Today, surplus food redistribution is coordinated mainly through phone calls, messaging groups and social media posts. A restaurant manager calls a known NGO, a volunteer forwards the message to a group, somebody replies that they can collect it, and the details are noted on paper or in a spreadsheet. This approach works only as long as the same few people are available and responsive. It offers no reliable record of what was donated, when it was collected, who received it and whether it reached the intended beneficiaries. It also makes it impossible to prepare the reports that NGOs, donors and regulators need. The Food Safety and Standards Authority of India has recognised the need to formalise this activity through its regulations on the recovery and distribution of surplus food [17], which further increases the value of traceable records.

The Food Surplus Redistribution and Community Hunger Reduction Platform is a database-centred backend system designed to solve this coordination problem. It connects four kinds of participants on a single platform: food donors, NGOs, volunteers and beneficiaries. Donors publish what they have and when it expires. NGOs and volunteers see the donations that are near them and that match their needs. The system reserves the food, schedules a pickup, records the collection and finally records the distribution to beneficiaries. Each step is stored in a consistent, queryable form, so the whole journey can be monitored and analysed.

From an engineering perspective the project is also a practical demonstration of modern database systems and distributed backend development. The problem naturally needs several kinds of data stores. Donations, inventory and pickups need strict transactional guarantees, so a relational database such as PostgreSQL is the right choice. User profiles, preferences and activity feeds change shape frequently, which suits the document model of MongoDB. Matching a donation to an NGO's needs benefits from similarity search over embeddings, which is available through the pgvector extension. The backend itself is a good candidate for a microservice architecture, because user management, food listings and matching evolve at different speeds and have different scaling needs. For these reasons the project is built as a polyglot system using FastAPI, Node.js with Express, and Spring Boot, and it is packaged with Docker, orchestrated with Kubernetes and monitored with OpenTelemetry, Prometheus and Grafana.

The project therefore has two complementary backgrounds. The first is a social one: reducing food wastage and supporting communities in need. The second is an academic one: applying relational database engineering, NoSQL and vector databases, multi-framework backend development, microservices and DevOps practices to a realistic problem. The rest of this report explains how both objectives were met.

1.2 Problem Statement

Large quantities of edible food are wasted by restaurants, hotels, event organisers, supermarkets and households every day. At the same time, many people and communities face food shortages. There is no centralised and reliable system that connects the people who have surplus food with the organisations that can collect and distribute it. As a result, donation details are exchanged informally, pickups are delayed, quantities are recorded inconsistently and a large share of the surplus expires before it can be used.

The core problems can be stated as follows.

- **Lack of a central platform.** Donors, NGOs, volunteers and beneficiaries operate in isolation. Information about available food travels through calls and chat messages that are easily missed.
- **Time-critical spoilage.** Cooked and perishable food has a short safe-consumption window. Manual coordination is too slow to use that window well.
- **Poor data quality.** Donation details maintained on paper or in spreadsheets are prone to duplication, missing values and conflicting updates, for example two NGOs believing they have reserved the same tray of food.

- **No end-to-end traceability.** It is difficult to follow food from donor to pickup to NGO to beneficiary, which weakens accountability and donor trust.
- **Weak matching.** Even when information is available, it is hard to decide which NGO or volunteer should receive a particular donation, considering distance, quantity, food category and cold-chain needs.
- **No analytics.** Without structured data, it is impossible to report how many meals were saved, which areas are under-served or which donors contribute most.

From the engineering point of view, the problem is to design and implement a backend that guarantees consistent inventory and pickup records under concurrent use, handles heterogeneous data appropriately, exposes secure and well-documented APIs, scales each concern independently and can be deployed and monitored like a production system. This is the problem addressed by this project.

1.3 Scope of the Project

The scope of the project covers the design, implementation, testing and deployment of the backend platform and a supporting web interface for the complete surplus food redistribution workflow.

In scope

- User registration, login and role-based access for four roles: donor, NGO coordinator, volunteer and administrator.
- Donor management, including donor profiles and creation, update, cancellation and status tracking of surplus food listings with type, quantity, location, preparation time and expiry time.
- NGO and volunteer management, including NGO verification, volunteer enrolment and availability.
- Food request management, where NGOs state the category and quantity of food they need and the time by which they need it.
- Location-based and quantity-based matching of donations to NGO requests, together with a semantic similarity score generated from vector embeddings.
- Reservation, pickup scheduling, volunteer assignment, collection confirmation and distribution recording, protected by ACID transactions and Saga-based compensation.
- Real-time notifications, activity feeds, ratings and feedback.
- Reports and analytics using advanced SQL, including joins, subqueries, common table expressions, aggregates and window functions.

- A retrieval-augmented assistant that answers food-safety and handling questions from a curated knowledge base.
- Containerisation, Kubernetes manifests, observability dashboards and an automated CI/CD pipeline with security scanning.

Out of scope

- Online payments and monetary donations, because the platform concerns food redistribution only.
- Native mobile applications. The web interface is responsive and can be installed as a progressive web application.
- Automated food-quality inspection. The platform records donor declarations and follows safe-handling guidelines but does not certify food.
- Route optimisation with live traffic data, which is listed under future enhancements.

1.4 Objectives of the Project

The main objective is to build a reliable, secure and scalable platform that redistributes surplus food to people in need while demonstrating the engineering concepts of the course. The specific objectives are listed below.

1. To reduce food wastage by redistributing surplus food to NGOs and beneficiaries within its safe-consumption window.
2. To create a centralised, normalised relational database for donations, inventory, pickups and distributions, with constraints that prevent invalid or inconsistent data.
3. To connect donors, NGOs, volunteers and beneficiaries through a single role-based platform with clear ownership of every action.
4. To maintain accurate records of food quantity, category, expiry time and availability throughout the donation lifecycle.
5. To manage pickup and delivery efficiently by matching donations to nearby NGOs and assigning volunteers.
6. To track the complete donation and distribution journey, and keep an audit trail of every important change.
7. To generate useful reports and statistics using advanced SQL features such as joins, common table expressions and window functions.
8. To use each type of database where it fits best: PostgreSQL for transactions, MongoDB for flexible documents, pgvector for similarity search and Redis for caching and messaging.

9. To build RESTful APIs with FastAPI, Node.js with Express and Spring Boot, documented with OpenAPI and protected with JWT authentication, RBAC and rate limiting.
10. To decompose the system into independently deployable microservices with database-per-service ownership and to handle distributed consistency using Saga and compensating actions.
11. To containerise, deploy, monitor and continuously deliver the system using Docker, Kubernetes, OpenTelemetry, Prometheus, Grafana and GitHub Actions.
12. To improve the transparency, efficiency and reliability of food redistribution so that donors and communities can trust the process.

1.5 Importance of the Application

The importance of the application can be understood at social, economic, environmental and academic levels.

Social importance

The platform brings surplus food to people who need it. Even a modest number of successful redistributions provides real meals to families, shelters, orphanages and community kitchens. The traceability offered by the system also builds trust. Donors can see that their contribution reached beneficiaries, and NGOs can demonstrate their impact to supporters and authorities.

Economic importance

Food that is thrown away represents wasted money for restaurants, caterers and retailers, and wasted effort for the farmers and workers who produced it. Redistribution allows donors to convert a disposal cost into a social contribution. NGOs benefit because they can obtain food without spending on procurement, and can plan collection efficiently instead of relying on chance.

Environmental importance

Food that ends up in landfills decomposes and releases methane, a potent greenhouse gas. Preventing waste at the source reduces this burden and also saves the water, energy and land used to produce the food in the first place. The platform contributes to the global aim of halving per-capita food waste at retail and consumer levels by 2030, which is the target set out in Sustainable Development Goal 12.3 [16].

Technical and academic importance

The project shows how a real-world problem can be solved with a disciplined use of database and backend engineering. It demonstrates entity-relationship modelling, normalisation, indexing, transactions and stored logic on the relational side, document modelling and aggregation on the NoSQL side, vector search for semantic matching, layered API design in three frameworks, microservice decomposition, and a full delivery pipeline. The result is a compact but complete reference of how the six course outcomes fit together in one system.

1.6 Target Users / Audience

The platform is designed for the following groups of users. Each group has a dedicated role in the authorisation model, and the interface and the APIs expose only the operations that are relevant to that role.

Table 1.1: Target users and their goals

User group	Typical examples	What they need from the platform
Food donors	Restaurants, hotels, caterers, wedding and event organisers, canteens, supermarkets, households	Quickly publish surplus food, see when it is reserved and collected, and receive confirmation of distribution
NGOs	Food banks, community kitchens, shelters, orphanages, charitable trusts	Discover nearby donations, raise food requests, reserve quantities and manage volunteers and beneficiaries
Volunteers	Individuals and student groups working with an NGO	View assigned pickups, navigate to the donor location, confirm collection and delivery
Beneficiaries	Families and communities served by NGOs	Receive food in time; they are recorded through their NGO and rate the service
Administrators	Platform staff and NGO network coordinators	Verify NGOs, monitor KPIs, resolve disputes and view audit trails and system health
Evaluators and researchers	Faculty, auditors, municipal bodies	Access reports on food saved, response times and unserved areas

1.7 Mapping of the Project to Course Outcomes

The course Database Systems Engineering and Distributed Backend Development is organised into six course outcomes (CO1 to CO6). The project was deliberately structured so that every outcome is exercised by an identifiable part of the system. Table 1.2 summarises this mapping, and the later chapters give the details.

Table 1.2: Mapping of course outcomes to project components

CO	Course outcome	Project component	Chapters
----	----------------	-------------------	----------

CO	Course outcome	Project component	Chapters
CO1	Relational Database Engineering	ER model, normalised PostgreSQL schema, constraints, indexes, advanced SQL, views, procedures, triggers, ACID and MVCC	5, 6
CO2	Database Engineering (SQL, NoSQL, Vector)	MongoDB documents and aggregation, polyglot persistence, pgvector similarity search, hybrid search and RAG assistant	3, 5, 6
CO3	Backend API Engineering with FastAPI	API Gateway and Matching service, Pydantic models, dependency injection, JWT, RBAC, rate limiting, SQLAlchemy, layered refactoring, pytest	3, 6, 8
CO4	Multi-Framework Backend Engineering	Node.js and Express user service with Socket.io and Redis; Spring Boot listing service with JPA and Spring Security; self-learning notes on Flask, Django and .NET Core	3, 6
CO5	Microservices Engineering	Service decomposition, database-per-service, API gateway, distributed concepts, Saga and compensating actions	4, 6
CO6	Deployment, Observability and Delivery	Docker and Compose, Kubernetes, OpenTelemetry, Prometheus, Grafana, GitHub Actions, image scanning, C4 diagrams and load testing	4, 8, 9

1.8 Organisation of the Report

The report follows the standard structure prescribed for project documentation. Chapter 1 introduces the background, problem, scope, objectives, importance and users. Chapter 2 lists the hardware and software requirements together with the functional and non-functional requirements. Chapter 3 describes the technology stack for the front end, the three back-end frameworks, the databases, version control and external services. Chapter 4 presents the high-level architecture, the layers and the deployment architecture. Chapter 5 documents the design, including the ER model, the relational schema, normalisation, indexing, the MongoDB and vector designs, and the API contracts. Chapter 6 explains the implementation module by module with representative code. Chapter 7 lists the features and explains how each of them works. Chapter 8 covers the testing strategy, tools and test cases. Chapter 9 describes deployment and environment configuration. Chapter 10 discusses the challenges faced and the current limitations. Chapter 11 outlines future enhancements. Chapter 12 concludes the report, and Chapter 13 lists the references.

CHAPTER - 2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The platform is designed to run on modest hardware during development and on commodity cloud virtual machines in production. Because the system is split into several services and several databases, the memory requirement is dominated by the number of containers that run at the same time. Table 2.1 lists the minimum and recommended configuration for a developer workstation that runs the complete stack with Docker Compose, and Table 2.2 lists the sizing used for a small production cluster.

Table 2.1: Hardware requirements for a development workstation

Component	Minimum	Recommended
Processor	Intel Core i3 (or equivalent) with 4 logical cores	Intel Core i5 / AMD Ryzen 5 or above with 8 logical cores
RAM	8 GB (4 GB is sufficient only for the databases and one service at a time)	16 GB, so that all services, databases and the monitoring stack run together
Storage	10 GB of free disk space	30 GB SSD, to hold container images, database volumes and logs
Network	Broadband internet connection for images, packages and external APIs	Stable broadband with at least 10 Mbps
Display	1366 x 768 resolution	1920 x 1080 resolution for dashboards and diagrams

Table 2.2: Sizing used for a small production deployment

Component	Configuration	Purpose
Application nodes	2 nodes, each with 2 vCPU and 4 GB RAM	Run the gateway and the three microservices with two replicas each
Database node	1 node with 2 vCPU, 8 GB RAM and 50 GB SSD	PostgreSQL (with pgvector) and MongoDB with persistent volumes
Cache and messaging	1 GB RAM instance	Redis for caching, rate limiting and Socket.io pub/sub
Monitoring node	1 vCPU and 2 GB RAM	Prometheus, Grafana and the OpenTelemetry Collector

End users need only a modern web browser on a laptop, tablet or smartphone and an internet connection. Volunteers who use the platform in the field require a smartphone with location services enabled, so that pickup addresses and navigation links can be opened directly.

2.2 Software Requirements

The software requirements are grouped by layer. Versions are indicated so that the environment can be reproduced exactly. The same versions are pinned in the Dockerfiles and in the Compose file described in Chapter 9.

Table 2.3: Software requirements

Category	Software	Version / Notes
Operating system	Windows 10/11, Ubuntu 22.04 LTS or macOS	Docker Desktop with WSL 2 on Windows
Relational database	PostgreSQL	16, with the pgvector, cube and earthdistance extensions
NoSQL database	MongoDB	7.x, accessed with Mongoose and Motor
Cache and messaging	Redis	7.x
Python runtime	Python	3.11 or above, with FastAPI, Pydantic v2, SQLAlchemy 2, Uvicorn
Node.js runtime	Node.js	20 LTS, with Express, Mongoose, Passport, Socket.io
Java runtime	JDK and Maven	JDK 17, Spring Boot 3.x, Maven 3.9
Containers	Docker Engine and Docker Compose	24 or above; Compose v2
Orchestration	Kubernetes (Minikube or kind for local testing)	1.28 or above, with kubectl
Observability	OpenTelemetry Collector, Prometheus, Grafana	Latest stable releases
Version control	Git and GitHub	Git 2.40 or above
Browser	Chrome, Edge, Firefox or Safari	Latest two versions

2.3 Development Tools and Frameworks

The team used a small set of well-supported tools so that each member could work on a different service without environment problems.

Table 2.4: Development tools and frameworks

Purpose	Tool or framework	How it was used
Code editing	Visual Studio Code, IntelliJ IDEA Community	Python and JavaScript in VS Code; Java in IntelliJ
Database tools	pgAdmin, psql, MongoDB Compass, DBeaver	Schema design, query tuning with EXPLAIN ANALYZE, document inspection
API testing	Postman and Swagger UI	Collections for every endpoint, exported as OpenAPI
Testing frameworks	pytest, Jest with Supertest, JUnit 5 with Testcontainers	Unit and integration tests per service
Load testing	k6 and Locust	Load scenarios for search and reservation endpoints

Purpose	Tool or framework	How it was used
Diagramming	draw.io, Graphviz, Mermaid	ER, C4 and architecture diagrams
CI/CD	GitHub Actions	Build, test, scan and publish images
Security scanning	Trivy, OWASP Dependency-Check, Bandit, npm audit	Image, dependency and static analysis
Project management	GitHub Projects and Issues	Task tracking and code review

2.4 Functional Requirements

The functional requirements describe what the system must do. They were derived from the problem statement and refined through discussions with the guide.

1. The system shall allow a user to register with one of the roles donor, NGO coordinator, volunteer or administrator, and shall authenticate users with a secure token-based mechanism.
2. The system shall allow a donor to create, update, view and cancel food listings with category, quantity, unit, preparation time, expiry time and pickup location.
3. The system shall automatically mark a listing as expired when its expiry time has passed.
4. The system shall allow an NGO to search available donations by distance, category and quantity, and to raise food requests stating the required quantity and the time by which it is needed.
5. The system shall match donations and requests and shall rank the candidate NGOs using distance, quantity fit and semantic similarity.
6. The system shall allow an NGO to reserve all or part of a donation, and shall guarantee that the reserved quantity never exceeds the available quantity, even when many NGOs reserve concurrently.
7. The system shall assign a volunteer to each reservation, allow the volunteer to confirm collection, and record the distribution to beneficiaries with the number of servings.
8. The system shall notify the relevant parties in real time about listing, reservation, assignment, collection and delivery events.
9. The system shall maintain an audit trail of changes to donations, reservations and pickups.
10. The system shall provide reports for administrators and NGOs, including food saved, average response time, top donors and under-served areas.
11. The system shall answer food-safety and handling questions using a curated knowledge base with source references.

2.5 Non-Functional Requirements

Table 2.5: Non-functional requirements

Attribute	Requirement	How it is addressed
Consistency	Inventory and pickup records must remain correct under concurrent updates	ACID transactions, row-level locking and CHECK constraints; Saga for cross-service work
Performance	Search and listing endpoints should respond within 300 ms at the 95th percentile under the target load	Indexing, Redis caching, connection pooling, pagination
Scalability	Each service must scale independently	Stateless services, Kubernetes Deployments, database-per-service
Availability	A failure in one service must not bring down the whole platform	Timeouts, retries with back-off, circuit breaking at the gateway
Security	Protect accounts, personal data and APIs against common attacks	Hashed passwords, JWT with short expiry, RBAC, rate limiting, input validation, OWASP guidelines
Observability	Operators must be able to see the health and behaviour of the system	Structured logs, metrics, distributed traces, dashboards and alerts
Usability	Interfaces must be simple for non-technical donors and volunteers	Responsive layout, minimal forms, clear status indicators
Maintainability	Code must be modular and testable	Layered architecture, dependency injection, automated tests and CI
Portability	The platform must run identically on any machine	Docker images and declarative configuration

CHAPTER - 3 TECHNOLOGY STACK

The technology stack was selected by applying one principle: each concern should be handled by the tool that suits it best, provided that the resulting system stays understandable and maintainable. This is the idea of polyglot persistence and polyglot services, and it also allows the team to practise all frameworks listed in the course. Figure 3.1 summarises where each kind of data lives, and the following sections describe every layer.

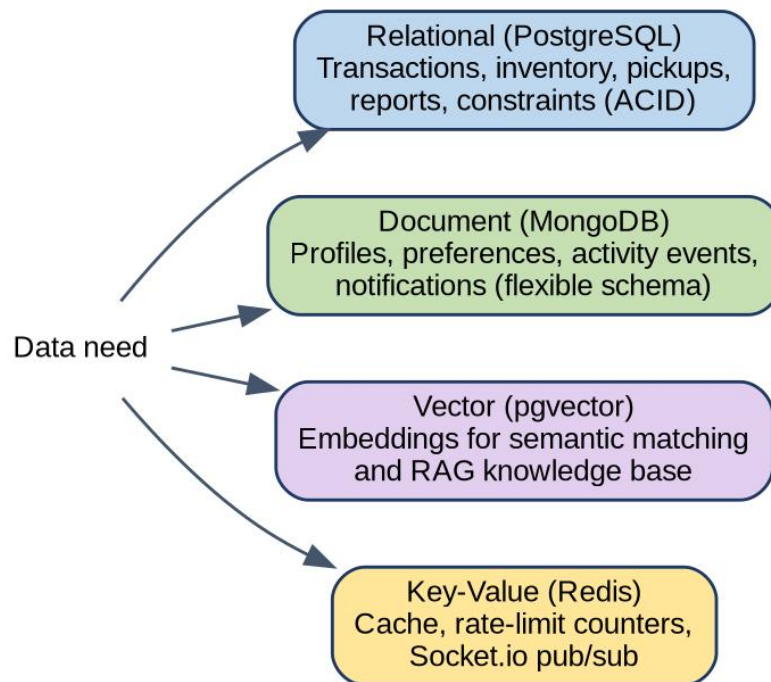


Figure 3.1: Polyglot persistence: data needs mapped to database types

3.1 Front-End Design

The front end is a single-page web application built with **React 18** and bundled with **Vite**. React's component model suits the platform because the same building blocks, such as a listing card, a status badge and a map preview, are reused across the donor, NGO, volunteer and administrator dashboards. **Tailwind CSS** provides utility classes for a consistent and responsive layout, so that the interface works on a laptop in an NGO office and on a phone in the field.

State management is deliberately simple. Server data is fetched and cached with **React Query**, which handles loading states, retries and background refresh. Local interface state uses React hooks, and authentication state is kept in a small context provider. Real-time updates arrive through a **Socket.io client**, which allows a donor to see the status of a listing change from AVAILABLE to RESERVED without refreshing the page. Maps are rendered with **Leaflet** using OpenStreetMap tiles, and charts on the administrator dashboard are drawn with **Recharts**.

Form validation is performed on the client with **Zod** schemas that mirror the server-side Pydantic and Bean Validation rules, so users receive immediate feedback while the server remains the final authority.

The interface follows three design rules. First, every screen shows the status of the current donation prominently, because timing is critical. Second, actions that are not permitted for the user's role are hidden, but the server enforces the same rules. Third, the application can be installed as a progressive web application, and it caches the shell so that volunteers with an unstable connection can still open their assigned pickups.

3.2 Back-End Design

The back end is deliberately multi-framework. The three main frameworks of the course, FastAPI, Node.js with Express and Spring Boot, are each used for the service where their strengths matter most. All services expose REST endpoints with JSON payloads, publish an OpenAPI description and read their configuration from environment variables.

3.2.1 FastAPI: API Gateway and Matching Service

FastAPI is a modern Python framework built on Starlette and Pydantic [19][20]. It provides automatic request validation from type hints, automatic OpenAPI documentation, a dependency injection system and first-class support for asynchronous code. These properties make it a good fit for two components. The **API Gateway** is an I/O-bound service that receives every client request, verifies the JWT, applies role checks and rate limiting, and forwards the request to the right service using an asynchronous HTTP client. The **Matching and Insight Service** performs scoring, vector similarity queries and analytical SQL, and benefits from Python's data and machine learning ecosystem. Both use **SQLAlchemy 2** for relational access, **Motor** for asynchronous MongoDB access where needed, and **pytest** for testing.

3.2.2 Node.js and Express: User and Activity Service

Node.js uses a single-threaded event loop with non-blocking I/O, which makes it well suited to services that handle many concurrent, lightweight connections [25]. The **User and Activity Service** issues and refreshes tokens, manages user profiles and produces the activity feed and notifications. It uses **Express** for routing and middleware, **Mongoose** for schema-based access to MongoDB [26], **Passport** with the JWT and local strategies for authentication, **bcrypt** for password hashing, **express-validator** for input validation, and **Socket.io** with a **Redis adapter** for real-time notifications that work across several service replicas [27]. Tests are written with Jest and Supertest.

3.2.3 Spring Boot: Food Listing and Pickup Service

Spring Boot brings the maturity of the Java ecosystem to the part of the system that needs the strongest transactional behaviour [28]. The **Food Listing and Pickup Service** owns donations, inventory and pickup orders. It uses Spring's inversion of control and dependency injection to wire controllers, services and repositories, **Spring Data JPA** with Hibernate for persistence, **Bean Validation** for request checking, **Spring Security** as an OAuth2 resource server that validates the JWT [29], **Spring Boot Actuator** for health and metrics, **springdoc-openapi** for API documentation and `@Async` tasks for non-blocking notification calls. Declarative transactions with `@Transactional` keep the database changes of each business operation atomic. Tests use JUnit 5, Mockito and Testcontainers.

3.2.4 Self-learning frameworks

The course also includes self-learning tracks on Flask, Django REST Framework and ASP.NET Core [43]. These frameworks are not used in the deployed system, but the team studied them and compared them with the chosen frameworks. Table 3.1 summarises the comparison and the reasoning that guided the final choice.

Table 3.1: Comparison of back-end frameworks studied during the project

Framework	Language	Strengths	Trade-offs	Role in this project
FastAPI	Python	Async support, automatic validation and OpenAPI, type hints	Younger ecosystem than Django	Gateway and Matching service
Flask	Python	Minimal, flexible micro-framework	Validation and docs need extensions	Studied; prototype of a health endpoint
Django + DRF	Python	Batteries included, admin site, mature ORM	Heavier, synchronous by default	Studied; compared for admin features
Express	JavaScript	Lightweight, huge middleware ecosystem, event-driven	Structure must be imposed by the team	User and Activity service
Spring Boot	Java	Strong typing, transactions, security, enterprise tooling	Higher memory use and start-up time	Food Listing and Pickup service
ASP.NET Core	C#	High performance, Identity, EF Core, Swagger	Requires the .NET toolchain	Studied; reference implementation of one endpoint

3.3 Database Design

The database layer is the heart of the project, and it was designed around the requirement that each type of data is stored in the model that suits it.

PostgreSQL 16 is the relational engine [22]. It stores donations, inventory, pickups, distributions and the master data of donors, NGOs, volunteers and beneficiaries. PostgreSQL provides ACID transactions, multi-version concurrency control (MVCC), rich constraints, advanced SQL features such as common table expressions and window functions, stored procedures, triggers and several index types (B-tree, GiST, GIN and HNSW). The **pgvector** extension adds a `vector` column type and approximate nearest-neighbour indexes, which lets the matching service run similarity search inside the same engine [23].

MongoDB 7 is the document database [24]. It stores user accounts and profiles, preferences, activity events and notifications. These documents differ between roles. A donor profile has business details, an NGO profile has service areas and certificates, and a volunteer profile has vehicle and availability information. A schema-flexible document model represents this variety naturally, and nested arrays such as saved addresses avoid unnecessary joins.

Redis 7 provides an in-memory key-value store for caching of frequent queries, rate-limit counters at the gateway and the pub/sub channel used by Socket.io.

SQL versus NoSQL: the decision framework

Table 3.2: Decision framework for choosing between relational and document storage

Criterion	PostgreSQL (relational)	MongoDB (document)	Decision in this project
Schema	Fixed, enforced by constraints	Flexible, validated optionally	Fixed for inventory; flexible for profiles
Relationships	Strong foreign keys and joins	Embedding or manual references	Foreign keys for donations and pickups
Transactions	Full ACID across tables	Multi-document transactions possible but costly	ACID for reservation and stock changes
Query style	SQL with joins, CTEs, windows	Aggregation pipeline	SQL for reports; pipeline for activity feed
Consistency (CAP)	Consistency-first (CP-leaning) on a single primary	Tunable consistency with replica sets	Strong consistency where money-like inventory is involved
Scaling	Vertical first, read replicas, partitioning	Horizontal sharding built in	Activity data can shard later

According to the CAP theorem, a distributed data store can guarantee at most two of consistency, availability and partition tolerance during a network partition [12]. For inventory the system chooses consistency, because promising the same food to two NGOs damages trust. For the activity feed the system accepts eventual consistency, because a notification that arrives a second late is harmless.

3.4 Version Control

Git and **GitHub** are used for version control and collaboration. The repository follows a mono-repository layout with one folder per service, a folder for infrastructure files and a folder for documentation. Development follows a simple branching model: the `main` branch is always deployable, work happens on short-lived `feature/*` branches, and changes reach `main` only through pull requests that pass the automated checks and receive at least one review. Commit messages follow the Conventional Commits style (for example `feat(listing): add reservation endpoint`), which makes the history easy to read and supports automatic change logs. Secrets are never committed. Configuration templates are stored as `.env.example` files, and real values are provided through environment variables and GitHub encrypted secrets.

Table 3.3: Repository layout

Folder	Contents
<code>gateway/</code>	FastAPI API Gateway
<code>matching-service/</code>	FastAPI Matching and Insight service
<code>user-service/</code>	Node.js and Express User and Activity service
<code>listing-service/</code>	Spring Boot Food Listing and Pickup service
<code>frontend/</code>	React web application
<code>db/</code>	SQL scripts: DDL, views, procedures, triggers, seed data; MongoDB init scripts
<code>deploy/</code>	Docker Compose file, Kubernetes manifests, Prometheus and Grafana configuration
<code>docs/</code>	C4 diagrams, ER diagrams, API contracts and this report
<code>.github/workflows/</code>	GitHub Actions pipelines

3.5 APIs or External Services

The platform uses a small number of external services, each behind an interface so that it can be replaced.

Table 3.4: External APIs and services

Service	Used for	Notes
OpenStreetMap Nominatim	Geocoding pickup addresses into latitude and longitude [42]	Public usage policy is respected with caching and rate limits; a self-hosted instance is the production option
OpenStreetMap tiles with Leaflet	Map display and navigation links	No API key required
SMTP provider	E-mail confirmations and password-reset messages	Configured through environment variables
SMS gateway	Optional text alerts to volunteers	Wrapped by an adapter, disabled in development

Service	Used for	Notes
Embedding model (Sentence-Transformers)	Convert donation descriptions and NGO needs into vectors [15]	The <code>all-MiniLM-L6-v2</code> model produces 384-dimensional embeddings and runs locally
Vector database options studied	Pinecone and Weaviate [44]	Studied as managed alternatives; pgvector was chosen to keep data in PostgreSQL
GitHub Container Registry	Storage of Docker images	Images are tagged with the commit SHA

Every internal service also exposes its own REST API. The gateway presents them to the outside world under a single base path such as `/api/v1`, and the OpenAPI documents of the individual services are merged into one interactive documentation page for developers.

CHAPTER - 4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The platform follows a microservice architecture behind a single API gateway. Figure 4.1 shows the complete system. Clients never talk directly to a service or a database. Every request enters through the FastAPI gateway, which authenticates and authorises it and forwards it to one of three domain services. Each service owns exactly one database, and no service reads another service's tables or collections. Data that is needed across services is exchanged through APIs or events. This rule, known as database-per-service, keeps the services loosely coupled and independently deployable [4][5].

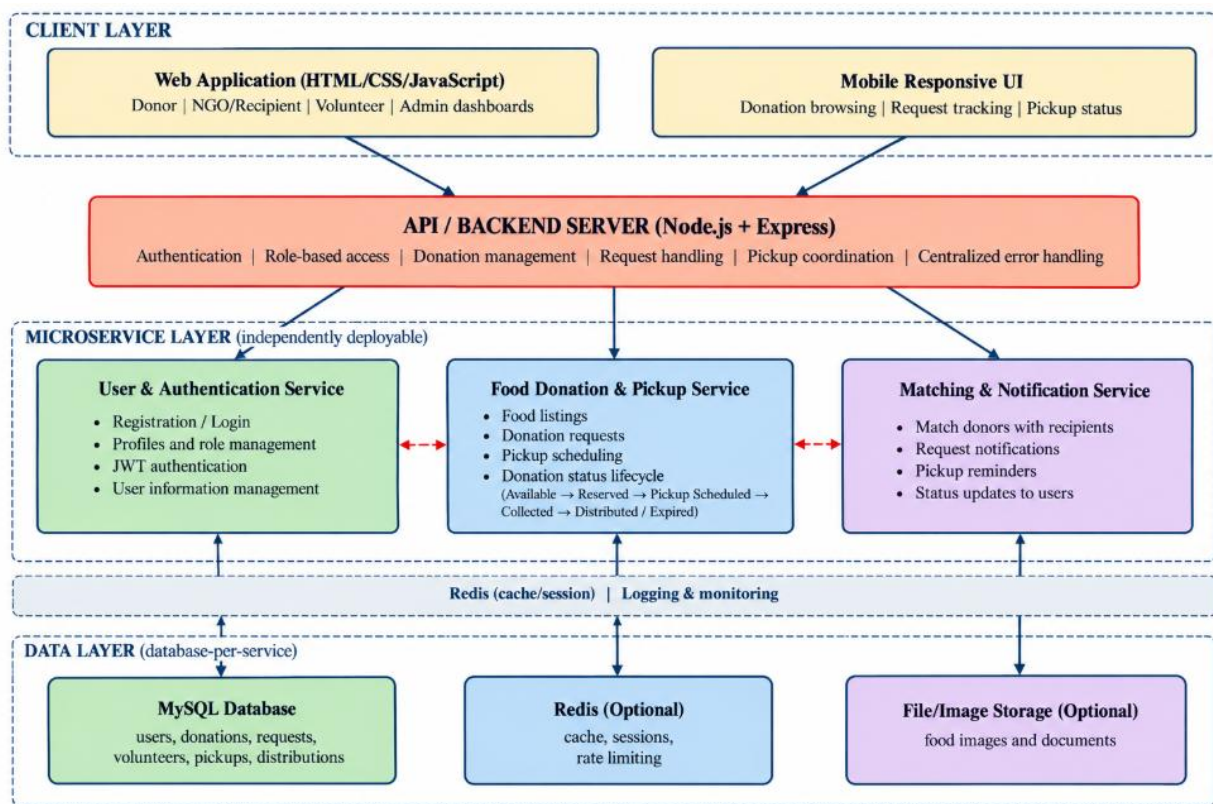


Figure 4.1: High-level architecture of the Food Surplus Redistribution Platform

The system was designed using the C4 model, which describes software at four zoom levels: context, containers, components and code [18]. Figure 4.2 shows the system context diagram. It identifies the people who use the platform and the external systems on which it depends. The container level is the architecture diagram shown in Figure 4.1, and the component level is shown for the FastAPI services in Section 4.2.

Service decomposition and boundaries

The decomposition follows business capabilities rather than technical layers. Three questions guided the choice of boundaries. First, which data changes together? Donations, inventory and

pickups must change in one transaction, so they belong to one service. Second, which parts have different scaling and change profiles? User profiles and notifications are read-heavy and change shape often, while matching is computationally heavy and depends on machine learning libraries. Third, which parts need different technology strengths? Table 4.1 summarises the result.

Table 4.1: Service responsibilities and data ownership

Service	Technology	Responsibilities	Owned data
API Gateway	FastAPI	Routing, JWT verification, RBAC, rate limiting, token forwarding, error normalisation, correlation IDs	None (Redis counters only)
User and Activity Service	Node.js, Express	Registration, login, token issue and refresh, profiles, activity feed, notifications, real-time push	MongoDB: users, activities, notifications
Food Listing and Pickup Service	Spring Boot	Donors, NGOs, volunteers, beneficiaries, listings, inventory, requests, pickups, distributions, Saga orchestration	PostgreSQL: surplus_db
Matching and Insight Service	FastAPI	Candidate ranking, vector search, RAG assistant, analytics and reports	PostgreSQL with pgvector: matching_db

Request flow

Figure 4.3 illustrates a typical request, the creation of a surplus listing and the events that follow it. The steps show how the layers cooperate. The gateway rejects invalid or unauthorised calls early, the Listing Service performs the transactional write, the Matching Service ranks nearby NGOs, and the User Service stores and pushes notifications. The NGO's reservation then starts the Saga described in Chapter 6.

4.2 Description of Each Layer (Frontend, Backend, Database)

Presentation layer

The React application is the only client. It is served as static files by an NGINX container and communicates with the platform exclusively through the gateway's `/api/v1` endpoints and the Socket.io endpoint for notifications. It holds no business rules other than input validation, and it never stores tokens in cookies that are accessible to scripts other than through the secure storage strategy described in Chapter 6.

API gateway layer

The gateway is the single entry point. It performs five duties. It **authenticates** requests by validating the JWT signature, expiry and issuer. It **authorises** requests by comparing the role

claim with the route's required role. It **rate-limits** clients using per-client counters in Redis so that abusive clients receive HTTP 429 responses. It **routes** the request to the correct service and **forwards the token** and a correlation identifier so that downstream services and traces can link the request. Finally it applies **centralised error handling**: errors from services, timeouts and connection failures are converted into one uniform JSON error format, so that clients handle failures in a single way.

Business service layer

Inside each service the code is organised into layers. Figure 4.4 shows the layered structure of the FastAPI services, which is also the structure the Spring Boot service follows with controllers, services and repositories. Routers handle HTTP details only. Dependencies provide authentication, authorisation and database sessions through dependency injection. Services contain business rules such as scoring and eligibility. Repositories contain database queries and nothing else. This separation allowed the team to refactor the first monolithic FastAPI prototype into services without rewriting the business logic, because each layer had a clear responsibility.

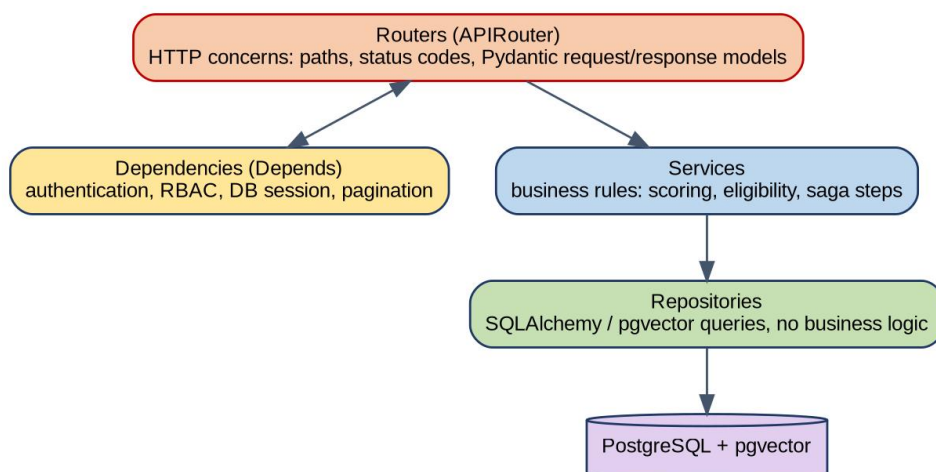


Figure 4.4: Layered structure of a FastAPI service

Data layer

The data layer consists of two PostgreSQL databases (`surplus_db` and `matching_db`, running on one server but logically isolated), one MongoDB database and one Redis instance. The relational schema follows the three-schema architecture of a database management system, which separates the external views seen by users, the conceptual schema of entities and constraints, and the internal storage structures [1][2]. Figure 4.5 shows how the platform uses each level. Views such as `my_donations_v` and `nearby_available_donations_v` give each role a tailored external schema. The conceptual schema is the normalised set of tables described in Chapter 5, and the internal schema consists of indexes, partitions and storage parameters that

can be tuned without changing the applications. This separation provides logical and physical data independence, which allowed the team to add indexes and columns during development without breaking the running services.

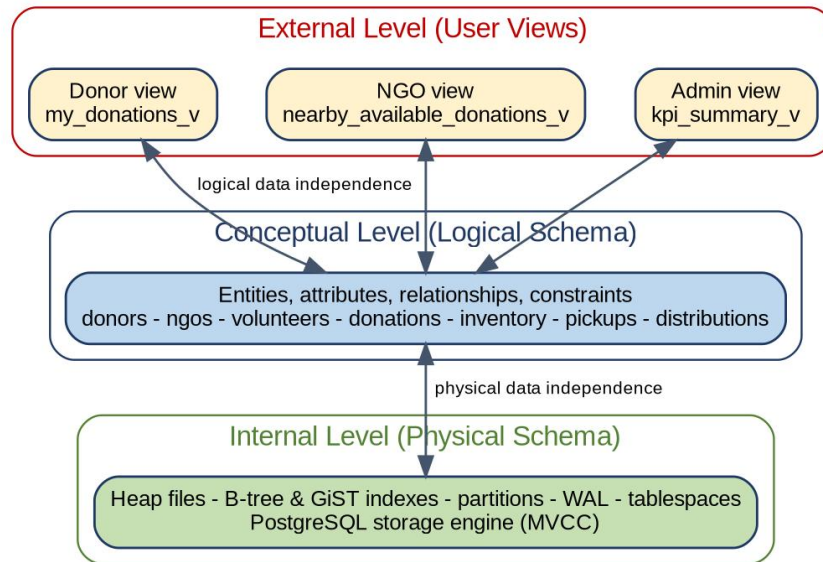


Figure 4.5: Three-schema architecture applied to the platform database

Cross-cutting concerns

Security, configuration, logging and observability cut across all layers. Every service reads configuration from environment variables, writes structured JSON logs that include the correlation identifier, exports traces and metrics through OpenTelemetry, and exposes `/health` and `/metrics` endpoints. Resilience is handled through timeouts on every outgoing call, limited retries with exponential back-off for idempotent requests, and a circuit breaker in the gateway that stops calling a failing service for a short cool-down period.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The same container images are used in every environment. In local development the complete stack, including databases and monitoring, starts with a single `docker compose up` command. In the cloud, the images are deployed to a Kubernetes cluster. Figure 4.6 shows the deployment architecture and the delivery path from a developer's commit to running Pods.

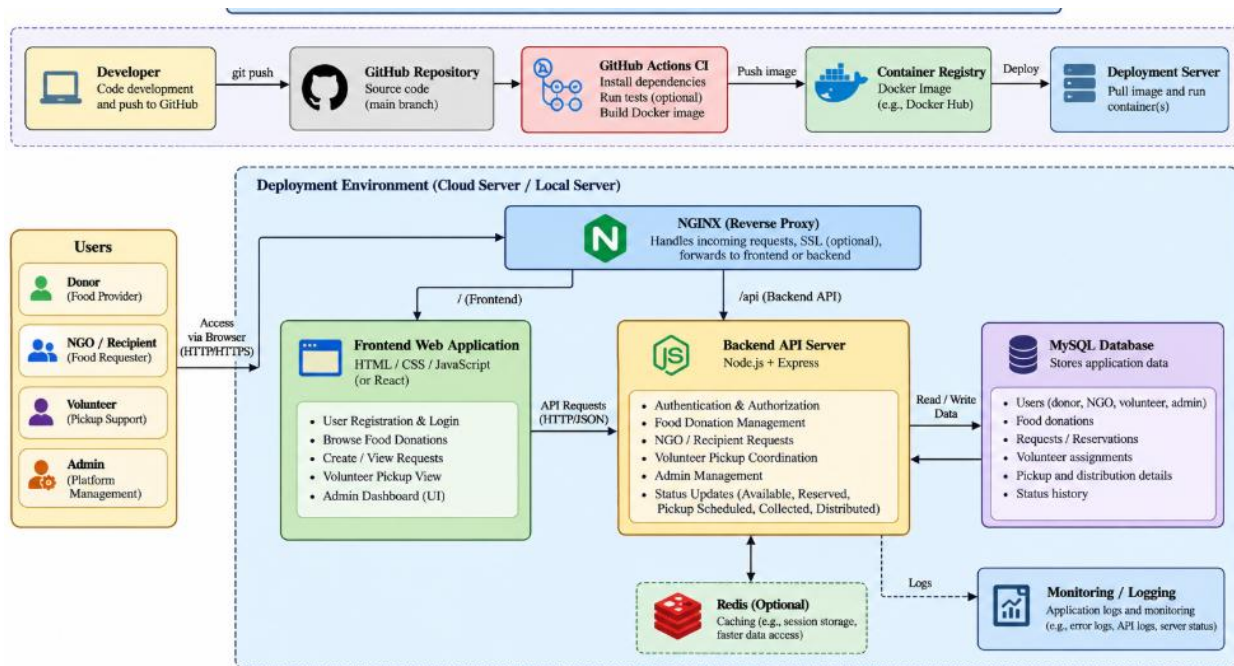


Figure 4.6: Deployment architecture with CI/CD pipeline and Kubernetes cluster

Local deployment with Docker Compose

Docker Compose defines one service per container, a private bridge network so that services reach each other by name, named volumes for database persistence, health checks that let dependent containers wait for the databases, and an `.env` file for configuration. Only the gateway, the front end and the Grafana dashboard publish ports to the host. The databases and internal services are reachable only inside the Compose network, which reduces the attack surface even in development.

Cloud deployment with Kubernetes

For the cloud target, each service runs as a Kubernetes **Deployment** with two replicas and readiness and liveness probes. Each Deployment is exposed inside the cluster by a **Service** of type ClusterIP. An **Ingress** controller terminates TLS and routes external traffic to the front end and the gateway. **ConfigMaps** and **Secrets** provide configuration, and the databases run as **StatefulSets** with persistent volume claims or are replaced by managed database services. Horizontal Pod Autoscalers can add replicas to the gateway and the matching service when CPU usage rises.

CI/CD pipeline

Every push and pull request triggers a GitHub Actions workflow with the following stages: dependency installation, linting, unit tests, integration tests with Testcontainers, image build, vulnerability scanning of the images with Trivy, dependency checks with OWASP Dependency-Check, and publication of the images to the container registry when the build runs on the `main`

branch. A failing stage stops the pipeline. Deployment to the cluster is performed by applying the versioned manifests, and a rollback is a single command that returns to the previous image tag.

Observability architecture

Figure 4.7 shows the observability pipeline. Every service is instrumented with OpenTelemetry and sends traces and metrics to an OpenTelemetry Collector, which forwards metrics to Prometheus and traces to a trace store. Grafana reads both and presents dashboards for request rate, error rate, latency percentiles, database connection usage and business indicators such as the number of active listings and the number of meals distributed per day. Alert rules notify the team when the error rate or latency crosses a threshold [32][33].

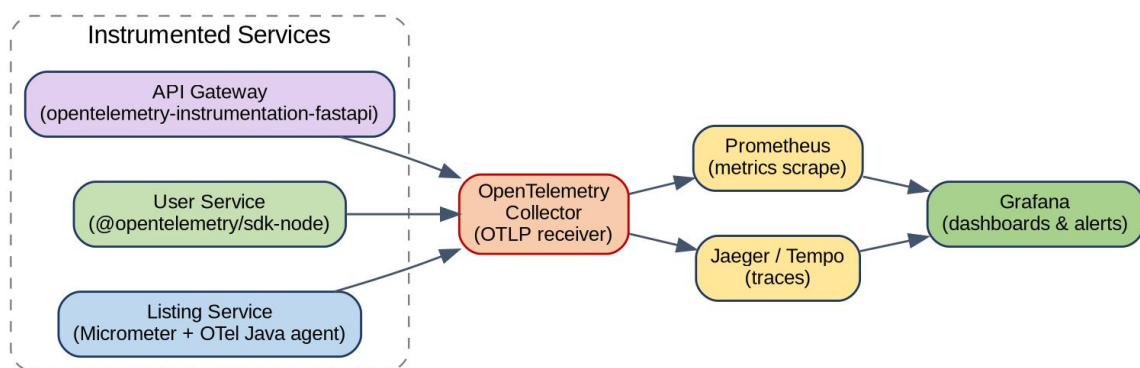


Figure 4.7: Observability pipeline: OpenTelemetry, Prometheus and Grafana

CHAPTER - 5 DESIGN

5.1 Design Overview and Use Cases

The design phase converted the requirements of Chapter 2 into a set of models: use cases, a donation lifecycle, an entity-relationship model, a relational schema, a document schema, a vector schema and API contracts. Design decisions were made in the order in which the data is created, starting from the people and organisations, then the food and its inventory, and finally the movement of the food.

Table 5.1: Principal use cases by actor

Actor	Use cases
Donor	Register; create listing; edit or cancel listing; view reservation and pickup status; view distribution confirmation; rate the service
NGO coordinator	Register and request verification; search nearby donations; raise food request; reserve donation; manage volunteers and beneficiaries; view reports
Volunteer	View assigned pickups; navigate to donor; confirm collection; confirm delivery; update availability
Administrator	Verify NGOs; monitor KPIs and audit log; suspend accounts; view system health dashboards
System (scheduler)	Expire stale listings; release timed-out reservations; refresh materialised reports; send reminders

Donation lifecycle

The most important design artefact is the donation lifecycle in Figure 5.1. A donation is always in exactly one state. The transitions are enforced in three places: a CHECK constraint restricts the state to the allowed values, a trigger rejects illegal transitions (for example DISTRIBUTED back to AVAILABLE), and the Saga orchestrator in the Listing Service is the only component that requests transitions. Expiry is a special transition. A scheduled job marks AVAILABLE listings as EXPIRED once `expires_at` has passed, so that stale food is never offered to NGOs.

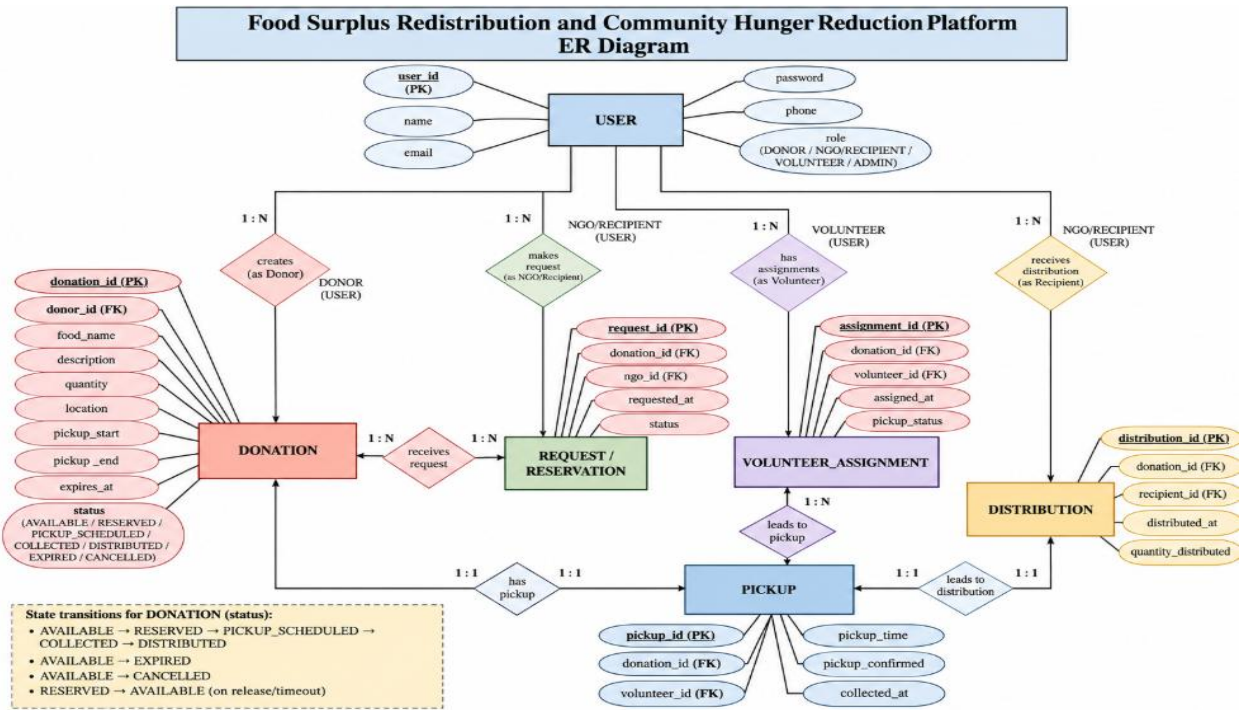


Figure 5.1: Donation status state machine

5.2 ER Diagram / Database Schema

Entity-relationship model

The entity-relationship model was drawn following Chen's notation adapted to a crow's foot style [10]. Figure 5.2 presents the relational core owned by the Food Listing and Pickup Service. The main entities and their roles are described in Table 5.2.

Table 5.2: Entities, purpose and key attributes

Entity	Purpose	Key attributes
donors	Organisations or individuals who give surplus food	donor_id, donor_type, org_name, address, latitude, longitude, user_ref
ngos	Verified organisations that collect and distribute food	ngo_id, reg_number, service_area, capacity_kg, verified
volunteers	People who collect and deliver food for an NGO	volunteer_id, ngo_id, vehicle_type, availability, rating_avg
beneficiaries	Communities or groups served by an NGO	beneficiary_id, ngo_id, community_name, headcount
food_categories	Reference data: cooked meal, bakery, fruit and vegetables, packaged food	category_id, name, shelf_life_hrs, needs_cold_chain
donations	A surplus listing published by a donor	donation_id, donor_id, category_id, quantity, unit, prepared_at, expires_at, status
inventory	Live stock counters for one donation	donation_id, total_qty, available_qty, reserved_qty, distributed_qty
food_requests	An NGO's stated need	request_id, ngo_id, category_id, required_qty, needed_by, status
pickups	A reservation and its collection task	pickup_id, donation_id, request_id, volunteer_id, reserved_qty, status

Entity	Purpose	Key attributes
distributions	Delivery of collected food to a beneficiary	distribution_id, pickup_id, beneficiary_id, servings, delivered_at
ratings	Feedback on a completed distribution	rating_id, distribution_id, score, comments
audit_log	Immutable history of changes	log_id, table_name, record_id, action, old_value, new_value

Cardinality decisions

A donor can publish many donations, but each donation belongs to exactly one donor (1:N). A donation has exactly one inventory row (1:1), which is kept separate from the donation so that the frequently updated stock counters do not lock or bloat the descriptive donation row. A donation can be split among several NGOs, so it can have many pickups (1:N), and a pickup may be linked to the NGO's request that it satisfies. An NGO enrolls many volunteers and serves many beneficiaries (both 1:N). A pickup can be delivered to several beneficiary groups, so it can have many distributions (1:N), and each distribution can have at most one rating (1:0..1). There are no many-to-many relationships left unresolved, because the pickup and distribution entities act as the associative entities between donations, volunteers and beneficiaries.

ER-to-relational mapping

The mapping followed the standard rules [1]. Every strong entity became a table with its identifier as the primary key. Every 1:N relationship was implemented by placing the primary key of the "one" side as a foreign key on the "many" side. The 1:1 relationship between donations and inventory was implemented by sharing the primary key. Associative entities kept foreign keys to both parents. Multi-valued attributes, such as the several food items in a mixed donation, are represented by separate donation lines in the design extension discussed in Chapter 11, and the current model uses one category per listing for clarity.

Relational schema (DDL)

The following statements show the core of the schema. Constraints express the business rules directly in the database, so that no application bug can store an invalid state.

```
CREATE EXTENSION IF NOT EXISTS cube;
CREATE EXTENSION IF NOT EXISTS earthdistance;

CREATE TYPE donation_status AS ENUM
('AVAILABLE', 'RESERVED', 'PICKUP_SCHEDULED', 'COLLECTED',
'DISTRIBUTED', 'EXPIRED', 'CANCELLED');

CREATE TABLE food_categories (
category_id SERIAL PRIMARY KEY,
name VARCHAR(60) NOT NULL UNIQUE,
shelf_life_hrs INT NOT NULL CHECK (shelf_life_hrs > 0),
needs_cold_chain BOOLEAN NOT NULL DEFAULT FALSE
);
```

```

CREATE TABLE donors (
  donor_id      BIGSERIAL PRIMARY KEY,
  user_ref      CHAR(24) NOT NULL UNIQUE,    -- ObjectId of the MongoDB user
  donor_type    VARCHAR(20) NOT NULL
               CHECK (donor_type IN ('RESTAURANT','HOTEL','EVENT',
                                     'SUPERMARKET','HOUSEHOLD')),
  org_name      VARCHAR(120) NOT NULL,
  address       TEXT NOT NULL,
  latitude      NUMERIC(9,6) NOT NULL CHECK (latitude BETWEEN -90 AND 90),
  longitude     NUMERIC(9,6) NOT NULL CHECK (longitude BETWEEN -180 AND 180),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE ngos (
  ngo_id        BIGSERIAL PRIMARY KEY,
  user_ref      CHAR(24) NOT NULL UNIQUE,
  name          VARCHAR(120) NOT NULL,
  reg_number    VARCHAR(40) NOT NULL UNIQUE,
  service_area  VARCHAR(120) NOT NULL,
  latitude      NUMERIC(9,6) NOT NULL,
  longitude     NUMERIC(9,6) NOT NULL,
  capacity_kg   NUMERIC(8,2) NOT NULL CHECK (capacity_kg >= 0),
  verified      BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE volunteers (
  volunteer_id  BIGSERIAL PRIMARY KEY,
  user_ref      CHAR(24) NOT NULL UNIQUE,
  ngo_id        BIGINT NOT NULL REFERENCES ngos(ngo_id),
  vehicle_type  VARCHAR(20) NOT NULL DEFAULT 'BIKE',
  availability   BOOLEAN NOT NULL DEFAULT TRUE,
  rating_avg    NUMERIC(3,2) CHECK (rating_avg BETWEEN 1 AND 5)
);

CREATE TABLE beneficiaries (
  beneficiary_id BIGSERIAL PRIMARY KEY,
  ngo_id         BIGINT NOT NULL REFERENCES ngos(ngo_id),
  community_name VARCHAR(120) NOT NULL,
  headcount      INT NOT NULL CHECK (headcount > 0),
  address        TEXT NOT NULL
);

```

```

CREATE TABLE donations (
  donation_id   BIGSERIAL PRIMARY KEY,
  donor_id      BIGINT NOT NULL REFERENCES donors(donor_id),
  category_id   INT NOT NULL REFERENCES food_categories(category_id),
  title         VARCHAR(120) NOT NULL,
  description    TEXT,
  quantity      NUMERIC(10,2) NOT NULL CHECK (quantity > 0),
  unit          VARCHAR(12) NOT NULL CHECK (unit IN ('KG','LITRE','MEALS','PACKS')),
  prepared_at   TIMESTAMPTZ NOT NULL,
  expires_at    TIMESTAMPTZ NOT NULL,
  pickup_lat    NUMERIC(9,6) NOT NULL,
  pickup_lng    NUMERIC(9,6) NOT NULL,
  status        donation_status NOT NULL DEFAULT 'AVAILABLE',
  version       INT NOT NULL DEFAULT 0,        -- optimistic locking
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (expires_at > prepared_at)
);

CREATE TABLE inventory (
  donation_id   BIGINT PRIMARY KEY REFERENCES donations(donation_id)
               ON DELETE CASCADE,
  total_qty     NUMERIC(10,2) NOT NULL CHECK (total_qty > 0),
  available_qty NUMERIC(10,2) NOT NULL CHECK (available_qty >= 0),
  reserved_qty  NUMERIC(10,2) NOT NULL CHECK (reserved_qty >= 0),
  distributed_qty NUMERIC(10,2) NOT NULL CHECK (distributed_qty >= 0),
  CHECK (available_qty + reserved_qty + distributed_qty = total_qty)
);

CREATE TABLE food_requests (

```

```

    request_id    BIGSERIAL PRIMARY KEY,
    ngo_id        BIGINT NOT NULL REFERENCES ngos(ngo_id),
    category_id   INT     NOT NULL REFERENCES food_categories(category_id),
    required_qty  NUMERIC(10,2) NOT NULL CHECK (required_qty > 0),
    needed_by     TIMESTAMPTZ NOT NULL,
    status        VARCHAR(12) NOT NULL DEFAULT 'OPEN'
                CHECK (status IN ('OPEN', 'MATCHED', 'FULFILLED', 'CANCELLED'))
);

CREATE TABLE pickups (
    pickup_id     BIGSERIAL PRIMARY KEY,
    donation_id   BIGINT NOT NULL REFERENCES donations(donation_id),
    request_id    BIGINT REFERENCES food_requests(request_id),
    volunteer_id  BIGINT REFERENCES volunteers(volunteer_id),
    reserved_qty  NUMERIC(10,2) NOT NULL CHECK (reserved_qty > 0),
    scheduled_at  TIMESTAMPTZ,
    picked_at     TIMESTAMPTZ,
    status        VARCHAR(12) NOT NULL DEFAULT 'PENDING'
                CHECK (status IN ('PENDING', 'SCHEDULED', 'COLLECTED',
                                'DELIVERED', 'CANCELLED'))
);

CREATE TABLE distributions (
    distribution_id BIGSERIAL PRIMARY KEY,
    pickup_id      BIGINT NOT NULL REFERENCES pickups(pickup_id),
    beneficiary_id BIGINT NOT NULL REFERENCES beneficiaries(beneficiary_id),
    servings        INT NOT NULL CHECK (servings > 0),
    delivered_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE ratings (
    rating_id      BIGSERIAL PRIMARY KEY,
    distribution_id BIGINT NOT NULL UNIQUE REFERENCES distributions(distribution_id),
    score          SMALLINT NOT NULL CHECK (score BETWEEN 1 AND 5),
    comments       TEXT
);

CREATE TABLE audit_log (
    log_id         BIGSERIAL PRIMARY KEY,
    table_name     VARCHAR(40) NOT NULL,
    record_id      BIGINT NOT NULL,
    action         VARCHAR(10) NOT NULL,
    old_value      JSONB,
    new_value      JSONB,
    changed_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

The inventory table deserves special mention. Its CHECK constraint states the invariant that available, reserved and distributed quantities always add up to the total. Any statement that would break this rule, for example a reservation larger than the available stock, fails inside the database and is rolled back, regardless of which service or script issued it.

5.3 Normalisation

Normalisation removes redundancy and update anomalies by ensuring that every non-key attribute depends on the key, the whole key and nothing but the key [1][2]. The team began with the single spreadsheet that an NGO typically keeps for donations and normalised it step by step.

Unnormalised form

A typical spreadsheet row contains: donation number, donor name, donor phone, donor address, food items (several items in one cell), quantities, expiry, NGO name, NGO address, volunteer

name, volunteer phone, pickup time, beneficiary community and servings. This layout has clear anomalies. The donor's phone number is repeated in every row, so a change must be made many times (update anomaly). A donor who has not yet donated cannot be recorded (insertion anomaly). Deleting the only donation of a donor deletes the donor's details as well (deletion anomaly).

First normal form (1NF)

Repeating groups such as the list of food items in one cell were removed, so that every attribute holds one atomic value and every row can be identified by a key. Each food item became a separate row identified by donation number and item.

Second normal form (2NF)

In the 1NF relation the key is (donation number, item). Attributes such as donor name, donor phone and expiry depend only on the donation number, which is a part of the key, so they show partial dependency. They were moved to a separate donation relation. Item-specific attributes stayed with the item.

Third normal form (3NF)

In the donation relation, donor phone and donor address depend on the donor and not directly on the donation, which is a transitive dependency (donation to donor to donor phone). The donor attributes moved into the `donors` table. The same step created `ngos`, `volunteers` and `beneficiaries` as separate tables and replaced their repeated details with foreign keys.

Boyce-Codd normal form (BCNF)

The functional dependencies of the final schema were checked one table at a time. For each dependency X to Y, X is a candidate key of its table. In `ngos` both `ngo_id` and `reg_number` are candidate keys, and every other attribute depends on them. In `inventory` the key is `donation_id`. No table contains a determinant that is not a key, so the schema is in BCNF.

Table 5.3: Functional dependencies of the main tables

Table	Determinant (candidate key)	Determined attributes
donors	donor_id; user_ref	donor_type, org_name, address, latitude, longitude
ngos	ngo_id; reg_number; user_ref	name, service_area, latitude, longitude, capacity_kg, verified
donations	donation_id	donor_id, category_id, title, quantity, unit, prepared_at, expires_at, status
inventory	donation_id	total_qty, available_qty, reserved_qty, distributed_qty

Table	Determinant (candidate key)	Determined attributes
pickups	pickup_id	donation_id, request_id, volunteer_id, reserved_qty, scheduled_at, status
food_categories	category_id; name	shelf_life_hrs, needs_cold_chain

Controlled denormalisation

Two deliberate exceptions exist. First, `volunteers.rating_avg` stores a value that could be computed from the `ratings` table. It is maintained by a trigger, because volunteers are ranked frequently and recomputing the average for every search would be expensive. Second, the reporting queries use a materialised view that stores daily totals. Both are documented, refreshed automatically and never edited by hand.

5.4 Index Design

Indexes were designed from the queries the application actually runs, not added blindly. The most frequent read is "which available donations are near this NGO and not yet expired", and the most frequent write is the update of inventory counters during reservation. Table 5.4 lists the indexes and their justification.

Table 5.4: Index design

Index	Type	Supports
donations(status, expires_at) WHERE status = 'AVAILABLE'	Partial B-tree	Fast listing of currently available donations; the index stays small because it excludes finished donations
donations using ll_to_earth(pickup_lat, pickup_lng)	GiST	Radius search (earth_box) for nearby donations
donations(donor_id, created_at DESC)	B-tree	A donor's own listings sorted by newest first
pickups(volunteer_id, status)	Composite B-tree	A volunteer's active tasks
food_requests(category_id, needed_by) WHERE status = 'OPEN'	Partial B-tree	Matching open requests by category and deadline
audit_log(table_name, record_id, changed_at DESC)	Composite B-tree	History of one record
donation_embeddings(embedding)	HNSW (vector_cosine_ops)	Approximate nearest-neighbour search in the matching database

```
CREATE INDEX idx_don_available ON donations (status, expires_at)
WHERE status = 'AVAILABLE';
CREATE INDEX idx_don_geo ON donations
USING gist (ll_to_earth(pickup_lat, pickup_lng));
CREATE INDEX idx_don_donor ON donations (donor_id, created_at DESC);
CREATE INDEX idx_pick_vol ON pickups (volunteer_id, status);
CREATE INDEX idx_req_open ON food_requests (category_id, needed_by)
WHERE status = 'OPEN';
CREATE INDEX idx_audit_rec ON audit_log (table_name, record_id, changed_at DESC);
```

Each index was validated with `EXPLAIN (ANALYZE, BUFFERS)`. Before the partial index existed, the planner scanned the whole donations table for the available-food query. After adding it, the planner used an index scan restricted to live listings. Because every index slows down writes and consumes memory, indexes on rarely filtered columns were rejected.

5.5 Views, Stored Logic and Transaction Design

Business logic that must never be bypassed is placed in the database. The design contains the following objects, whose code is shown in Chapter 6.

- **Views** provide the external schema. `nearby_available_donations_v` combines donations, inventory, categories and donors for NGOs. `my_donations_v` exposes a donor's own listings. `kpi_summary_v` aggregates meals distributed, response times and expiry rates for administrators.
- **Stored procedures** implement operations that must be atomic, such as `reserve_donation`, which locks the inventory row, checks the available quantity and creates the pickup.
- **Triggers** maintain the audit log, validate status transitions and update `rating_avg`.
- **Transactions** use the READ COMMITTED isolation level by default. Reservation uses an explicit row lock (`SELECT ... FOR UPDATE`) on the inventory row, so concurrent reservations of the same donation are serialised. PostgreSQL's multi-version concurrency control lets readers continue to see a consistent snapshot without waiting for writers, which keeps search fast while reservations are in progress [22].

5.6 MongoDB Schema Design

The User and Activity Service stores three collections. The decisions on embedding and referencing follow the access patterns: data that is always read together is embedded, and data that grows without bound or is queried independently is referenced.

Table 5.5: MongoDB collections and modelling decisions

Collection	Modelling approach	Indexes
users	One document per user. Role-specific profile is embedded as a sub-document. Saved addresses are an embedded array (bounded)	Unique on email; 2dsphere on profile.location
activities	One document per event (append-only). References the user by id because the number of events is unbounded	Compound on (userId, createdAt DESC); TTL for old debug events
notifications	One document per notification with read flag	Compound on (userId, read, createdAt DESC); TTL index that removes documents after 30 days

```
{
  "_id": "665f1c2e9b1e8a3d4c7a1b20",
  "email": "manager@spicegarden.example",
  "passwordHash": "$2b$12$...",
  "role": "DONOR",
  "profile": {
    "displayName": "Spice Garden Restaurant",
    "phone": "+91-90000-00000",
    "location": { "type": "Point", "coordinates": [78.4867, 17.4948] },
    "addresses": [ { "label": "Kitchen", "line1": "Road No. 2", "city": "Hyderabad" } ],
    "preferences": { "notifyBy": ["push", "email"], "typicalFood": ["cooked meals"] }
  },
  "surplusRef": { "donorId": 42 },
  "createdAt": "2026-08-27T10:15:00Z"
}
```

The field `surplusRef.donorId` links the document to the relational row, and the reverse link is `donors.user_ref`. Because the two databases cannot enforce a foreign key across engines, the link is created by the registration workflow and verified by a nightly reconciliation job, as explained under consistency strategies in Chapter 6.

5.7 Vector Database Design

Keyword filters cannot express that "leftover vegetable biryani, 30 plates" is a good match for an NGO request such as "cooked rice meals for a shelter". To capture such similarity, the matching service converts descriptions into **embeddings**, which are fixed-length numeric vectors produced by a language model so that semantically similar texts have nearby vectors [15]. Similarity can be measured by cosine distance, Euclidean (L2) distance or inner product. The platform uses cosine distance because it ignores vector length and compares direction, which suits text embeddings.

An exact search compares the query with every stored vector, which becomes slow as data grows. **Approximate nearest neighbour (ANN)** indexes trade a small loss of recall for a large gain in speed. Two index families are available in pgvector. **IVF (inverted file)** clusters vectors into lists and searches only the closest lists (controlled by `probes`). **HNSW (Hierarchical Navigable Small World)** builds a multi-layer graph in which searches start at a sparse top layer and descend to a dense bottom layer [13]. HNSW usually gives better recall at low latency and needs no training step, so it was chosen. Its parameters are `m` (links per node), `ef_construction` (build-time search width) and `ef_search` (query-time search width).

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE donation_embeddings (
  donation_id BIGINT PRIMARY KEY,
  embedding    vector(384) NOT NULL,
  text_hash    CHAR(64) NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```

CREATE INDEX ON donation_embeddings
  USING hnsw (embedding_vector_cosine_ops) WITH (m = 16, ef_construction = 64);

CREATE TABLE need_embeddings (          -- read model of open NGO requests
  request_id  BIGINT PRIMARY KEY,
  ngo_id      BIGINT NOT NULL,
  category_id INT NOT NULL,
  required_qty NUMERIC(10,2) NOT NULL,
  needed_by   TIMESTAMPTZ NOT NULL,
  latitude    NUMERIC(9,6) NOT NULL,
  longitude   NUMERIC(9,6) NOT NULL,
  status      VARCHAR(12) NOT NULL DEFAULT 'OPEN',
  embedding   vector(384) NOT NULL
);
CREATE INDEX ON need_embeddings USING hnsw (embedding_vector_cosine_ops);
CREATE INDEX ON need_embeddings USING gist (ll_to_earth(latitude, longitude));

CREATE TABLE knowledge_chunks (        -- RAG knowledge base
  chunk_id  BIGSERIAL PRIMARY KEY,
  source    VARCHAR(200) NOT NULL,
  content   TEXT NOT NULL,
  embedding vector(384) NOT NULL,
  tsv       tsvector GENERATED ALWAYS AS (to_tsvector('english', content)) STORED
);
CREATE INDEX ON knowledge_chunks USING hnsw (embedding_vector_cosine_ops);
CREATE INDEX ON knowledge_chunks USING gin (tsv);

```

The table `need_embeddings` is a read model. Because each service owns its data, the Matching Service does not read the Listing Service's tables. Instead, the Listing Service sends every new or changed NGO request to the Matching Service, which stores the fields needed for ranking together with the embedding.

For the food-safety assistant, the platform uses **hybrid search**. A query is run both as a full-text search over the `tsv` column and as a vector search over `embedding`. The two ranked lists are combined by reciprocal rank fusion, so exact terms such as "cold chain" and semantically related passages both contribute. The best passages are passed as context to a language model, which writes an answer that cites its sources. This pattern is called retrieval-augmented generation (RAG) [14].

5.8 API Contract Design

The APIs follow the REST constraints described by Fielding: client-server separation, statelessness, cacheability, a uniform interface, a layered system and, optionally, code on demand [8]. Resources are nouns in plural form, HTTP methods express the operation, and status codes follow HTTP semantics. Every endpoint is described in an OpenAPI 3 document [38], which acts as the contract between the front end and the services. The contract was agreed before implementation so that the teams could work in parallel.

Table 5.6: Principal REST resources exposed through the gateway

Method and path	Role	Description
POST /api/v1/auth/register	Public	Create an account for a chosen role

Method and path	Role	Description
POST /api/v1/auth/login	Public	Return access and refresh tokens
POST /api/v1/donations	DONOR	Create a surplus listing
GET /api/v1/donations?lat=&lng=&radius=&category=	NGO, VOLUNTEER	Search available donations nearby
PATCH /api/v1/donations/{id}	DONOR	Update or cancel a listing
POST /api/v1/requests	NGO	Raise a food request
POST /api/v1/donations/{id}/reservations	NGO	Reserve a quantity (starts the Saga)
GET /api/v1/pickups/mine	VOLUNTEER	List assigned pickups
POST /api/v1/pickups/{id}/collect	VOLUNTEER	Confirm collection
POST /api/v1/pickups/{id}/distributions	VOLUNTEER, NGO	Record delivery to a beneficiary
GET /api/v1/matches/{donationId}	NGO, ADMIN	Ranked candidate NGOs for a donation
POST /api/v1/assistant/ask	Any authenticated	Food-safety question answered with RAG
GET /api/v1/reports/summary	ADMIN, NGO	KPIs and trends
GET /api/v1/notifications	Any authenticated	Notifications of the current user

All errors use one JSON envelope so that clients can handle failures uniformly:

```
{
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "Requested 40 MEALS but only 25 are available.",
    "correlationId": "7c1f0d9e-3b0a-4a75-9c1e-0a3a2f1d5b77",
    "timestamp": "2026-09-02T09:31:12Z"
  }
}
```

State-changing operations that a client may retry, such as reservation, accept an **Idempotency-Key** header. If the same key arrives twice, the service returns the stored result instead of creating a second reservation. List endpoints are paginated with **page** and **size** parameters, and search endpoints cap the page size to protect the database.

CHAPTER - 6 IMPLEMENTATION

This chapter explains how the design was implemented. Sections 6.1 to 6.4 follow the structure prescribed for project reports (modules, front end, API integration, and authentication). Sections 6.5 to 6.9 describe the database programming, document and vector implementation, distributed consistency and the refactoring from monolith to services, which are the core of the course. The code fragments are representative extracts and have been shortened for readability.

6.1 Module-Wise Implementation

The platform is divided into seven modules. Table 6.1 shows the module, the service that implements it and the main technologies used.

Table 6.1: Modules and their implementation

Module	Service	Key implementation points
User authentication and management	User and Activity (Node.js)	Registration, bcrypt hashing, JWT (RS256) issue and refresh, profile documents in MongoDB
Food donor management	Food Listing and Pickup (Spring Boot)	Donor master data, listing creation, inventory initialisation in one transaction
Surplus food listing and monitoring	Food Listing and Pickup	Status state machine, expiry scheduler, optimistic locking with a version column
NGO and volunteer management	Food Listing and Pickup	NGO verification, volunteer enrolment, availability
Food request and matching	Matching and Insight (FastAPI)	Read model of requests, geo filter, pgvector similarity, weighted score
Collection and distribution	Food Listing and Pickup	Reservation with row locks, Saga orchestration, collection and distribution records
Notifications, reports and assistant	User and Activity; Matching and Insight	Socket.io push, aggregation pipelines, analytical SQL, RAG assistant

Food donor and listing module

The Listing Service exposes `POST /donations`. The controller validates the request with Bean Validation, the service creates the donation and its inventory row in one transaction, and the repository persists both. If either insert fails, neither is stored. After the transaction commits, the service publishes an event so that the Matching Service can compute the embedding and the User Service can record the activity.

```
@RestController
@RequestMapping("/donations")
@RequiredArgsConstructor
public class DonationController {
```

```

    private final DonationService service;

    @PostMapping
    @PreAuthorize("hasRole('DONOR')")
    public ResponseEntity<DonationDto> create(
        @Valid @RequestBody CreateDonationRequest req,
        @AuthenticationPrincipal Jwt jwt) {
        DonationDto created = service.create(jwt.getSubject(), req);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }
}

public record CreateDonationRequest(
    @NotBlank @Size(max = 120) String title,
    @NotNull Integer categoryId,
    @NotNull @DecimalMin("0.01") BigDecimal quantity,
    @NotNull Unit unit,
    @NotNull Instant preparedAt,
    @NotNull @Future Instant expiresAt,
    @NotNull @DecimalMin("-90") @DecimalMax("90") BigDecimal pickupLat,
    @NotNull @DecimalMin("-180") @DecimalMax("180") BigDecimal pickupLng) {}

@Service
@RequiredArgsConstructor
public class DonationService {
    private final DonationRepository donations;
    private final InventoryRepository inventories;
    private final DonorRepository donors;
    private final ApplicationEventPublisher events;

    @Transactional
    public DonationDto create(String userRef, CreateDonationRequest r) {
        Donor donor = donors.findByUserRef(userRef).orElseThrow(NotFoundException::new);
        Donation d = donations.save(Donation.from(donor, r));
        inventories.save(Inventory.full(d.getId(), r.quantity()));
        events.publishEvent(new DonationCreated(d.getId())); // sent after commit
        return DonationDto.of(d);
    }
}

```

Surplus listing monitoring and expiry

A scheduled task runs every minute inside the Listing Service. It expires stale listings with a single set-based statement rather than loading each row into memory, and the trigger described in Section 6.5 writes an audit record for every change.

```

@Scheduled(fixedDelay = 60_000)
@Transactional
public void expireStaleListings() {
    int n = jdbc.update("""
        UPDATE donations SET status = 'EXPIRED'
        WHERE status = 'AVAILABLE' AND expires_at <= now() """);
    if (n > 0) meter.counter("donations.expired").increment(n);
}

```

Matching module

The Matching Service ranks the NGO requests that can use a new donation. The score combines three signals: the semantic similarity between the donation text and the request text, the distance between the pickup location and the NGO, and the fit between the offered and the required quantity. The weights are configurable and were tuned with the guide's feedback. Section 6.7 shows the implementation.

6.2 Front-End Logic (UI Rendering and State Management)

The front end separates three kinds of state. **Server state** (donations, requests, pickups) is owned by React Query, which caches responses and refetches them in the background. **Session state** (the current user and tokens) lives in an authentication context. **Ephemeral UI state** (form inputs, open dialogs) stays inside the components that use it.

```
// hooks/useNearbyDonations.js
export function useNearbyDonations({ lat, lng, radiusKm = 5, category }) {
  return useQuery({
    queryKey: ['donations', lat, lng, radiusKm, category],
    queryFn: () => api.get('/donations', {
      params: { lat, lng, radius: radiusKm * 1000, category },
    }).then(res => res.data),
    enabled: lat != null && lng != null,
    staleTime: 15_000,
    refetchInterval: 60_000,
  });
}

// components/RealtimeBridge.jsx
export function RealtimeBridge() {
  const qc = useQueryClient();
  const { accessToken } = useAuth();
  useEffect(() => {
    if (!accessToken) return;
    const socket = io(import.meta.env.VITE_SOCKET_URL, { auth: { token: accessToken } });
    socket.on('notification', (n) => {
      toast(n.title);
      qc.invalidateQueries({ queryKey: ['donations'] }); // refresh statuses
      qc.invalidateQueries({ queryKey: ['pickups'] });
    });
    return () => socket.disconnect();
  }, [accessToken, qc]);
  return null;
}
```

Routes are protected by a small component that redirects unauthenticated users to the login page and hides pages that the user's role may not access. The server still enforces the same rule, so the client check exists only for usability. An Axios interceptor attaches the access token to every request and, when the gateway answers with HTTP 401, silently calls the refresh endpoint once and repeats the original request. If the refresh also fails, the user is logged out.

```
export function RequireRole({ roles, children }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  if (!roles.includes(user.role)) return <Navigate to="/403" replace />;
  return children;
}
```

Every list screen shows loading, empty and error states, and every donation card shows its remaining time in a colour that changes as the expiry approaches, so that NGOs can prioritise urgent food. Forms validate with Zod schemas before submission and display the field-level errors returned by the server in the common error format.

6.3 API Integration

The gateway is implemented in FastAPI. It wraps every request in a middleware that creates or propagates a correlation identifier, applies dependencies for rate limiting and role checks, and forwards the request with an asynchronous HTTP client that has explicit timeouts. Any error, whether it originates in the gateway or in a downstream service, is converted into the common error envelope.

```
# gateway/app/main.py
import uuid, httpx
from fastapi import FastAPI, Request, Depends, HTTPException
from fastapi.responses import JSONResponse
from .security import require_roles, rate_limit
from .config import SERVICES

app = FastAPI(title="Food Surplus API Gateway", version="1.0.0")
client = httpx.AsyncClient(timeout=httpx.Timeout(5.0, connect=2.0))

@app.middleware("http")
async def correlation_id(request: Request, call_next):
    cid = request.headers.get("X-Correlation-ID", str(uuid.uuid4()))
    request.state.cid = cid
    response = await call_next(request)
    response.headers["X-Correlation-ID"] = cid
    return response

def error(status: int, code: str, message: str, cid: str):
    body = {"error": {"code": code, "message": message, "correlationId": cid}}
    return JSONResponse(status_code=status, content=body)

@app.exception_handler(HTTPException)
async def http_error(request: Request, exc: HTTPException):
    return error(exc.status_code, f"HTTP_{exc.status_code}", str(exc.detail),
                request.state.cid)

async def forward(request: Request, service: str, path: str):
    headers = {"Authorization": request.headers.get("Authorization", ""),
               "Content-Type": request.headers.get("Content-Type", "application/json"),
               "X-Correlation-ID": request.state.cid}
    try:
        up = await client.request(request.method, SERVICES[service] + path,
                                   params=request.query_params,
                                   content=await request.body(), headers=headers)
    except httpx.TimeoutException:
        return error(504, "UPSTREAM_TIMEOUT", f"{service} timed out", request.state.cid)
    except httpx.ConnectError:
        return error(503, "UPSTREAM_UNAVAILABLE", f"{service} unavailable",
                    request.state.cid)
    content = up.json() if up.content else None
    return JSONResponse(status_code=up.status_code, content=content)

@app.post("/api/v1/donations",
          dependencies=[Depends(rate_limit), Depends(require_roles("DONOR"))])
async def create_donation(request: Request):
    return await forward(request, "listing", "/donations")

@app.get("/api/v1/donations",
          dependencies=[Depends(rate_limit), Depends(require_roles("NGO", "VOLUNTEER"))])
async def search_donations(request: Request):
    return await forward(request, "listing", "/donations")
```

Token forwarding means that the gateway does not create a new identity for downstream calls. The original JWT is passed on, and each service validates it again. This defence-in-depth

approach ensures that a service remains safe even if it is reached by a path that bypasses the gateway. Timeouts are short because a slow dependency should fail quickly rather than block the gateway's workers. Idempotent GET requests are retried once with a small delay, while non-idempotent POST requests are never retried automatically.

The service-to-service calls follow the same conventions. The Listing Service uses Spring's `RestClient` with connection and read timeouts to call the Matching Service and the User Service, and it propagates the correlation identifier in a header. The Matching Service exposes the following contract, which is documented automatically in its OpenAPI page:

```
# matching-service/app/routers/matches.py
router = APIRouter(prefix="/matches", tags=["matching"])

@router.post("/rank", response_model=list[MatchOut])
async def rank(body: RankRequest, svc: MatchingService = Depends(get_matching_service)):
    return await svc.rank_requests(body)          # donation -> best NGO requests

@router.post("/volunteers/rank", response_model=list[VolunteerScore])
async def rank_volunteers(body: VolunteerRankRequest,
                           svc: MatchingService = Depends(get_matching_service)):
    return svc.rank_volunteers(body)              # stateless scoring of candidates
```

6.4 Authentication and Authorisation

Authentication in the User Service

The User Service is the identity provider. Passwords are never stored in plain text. They are hashed with bcrypt using a cost factor of 12, which makes brute-force attacks expensive. Access tokens are JWTs signed with an RSA private key (RS256) [36], contain the user identifier as the subject and the role as a claim, and expire after 15 minutes. Longer-lived refresh tokens are stored hashed in MongoDB and can be revoked. Other services need only the public key to verify a token, so the signing key never leaves the User Service. Only the roles donor, NGO and volunteer can be chosen at registration. Administrator accounts are created by a seed script.

```
// user-service/src/routes/auth.js
router.post('/register',
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 8 }),
  body('role').isIn(['DONOR', 'NGO', 'VOLUNTEER']),
  async (req, res, next) => {
    const errors = validationResult(req);
    if (!errors.isEmpty())
      return res.status(422).json({
        error: { code: 'VALIDATION_FAILED', details: errors.array() } });
    try {
      const passwordHash = await bcrypt.hash(req.body.password, 12);
      const user = await User.create({ ...req.body, passwordHash });
      res.status(201).json({ id: user._id, role: user.role });
    } catch (e) {
      if (e.code === 11000)
        return res.status(409).json({ error: { code: 'EMAIL_EXISTS' } });
      next(e);
    }
  });
```

```

router.post('/login', async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  const ok = user && await bcrypt.compare(req.body.password, user.passwordHash);
  if (!ok) return res.status(401).json({ error: { code: 'BAD_CREDENTIALS' } });
  const accessToken = jwt.sign({ role: user.role }, process.env.JWT_PRIVATE_KEY, {
    algorithm: 'RS256', expiresIn: '15m', subject: String(user._id),
    issuer: 'user-service', audience: 'food-surplus' });
  res.json({ access_token: accessToken, token_type: 'bearer',
    refresh_token: await issueRefreshToken(user) });
});

```

Passport's JWT strategy protects the routes of the User Service itself. Login attempts are also limited per account and per address to slow down credential-stuffing attacks.

Verification and role-based access control in the gateway

The gateway verifies the token with the public key and then applies role-based access control (RBAC). The `OAuth2PasswordBearer` scheme is used so that the interactive Swagger page shows an Authorize button following the OAuth 2.0 password flow [37]. Rate limiting is implemented with an atomic counter in Redis, keyed by client address and minute.

```

# gateway/app/security.py
import time, jwt, redis.asyncio as redis
from fastapi import Depends, HTTPException, Request
from fastapi.security import OAuth2PasswordBearer
from .config import settings

oauth2 = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")
store = redis.from_url(settings.redis_url)

async def current_user(token: str = Depends(oauth2)) -> dict:
  try:
    return jwt.decode(token, settings.jwt_public_key, algorithms=["RS256"],
      audience="food-surplus", issuer="user-service")
  except jwt.ExpiredSignatureError:
    raise HTTPException(401, "Token expired")
  except jwt.InvalidTokenError:
    raise HTTPException(401, "Invalid token")

def require_roles(*roles: str):
  async def checker(user: dict = Depends(current_user)) -> dict:
    if user.get("role") not in roles:
      raise HTTPException(403, "Insufficient permissions")
    return user
  return checker

async def rate_limit(request: Request, limit: int = 100):
  key = f"rl:{request.client.host}:{int(time.time() // 60)}"
  count = await store.incr(key)
  if count == 1:
    await store.expire(key, 60)
  if count > limit:
    raise HTTPException(429, "Too many requests")

```

Verification in the Spring Boot service

The Listing Service is configured as an OAuth2 resource server. It validates the same JWT with the public key, converts the role claim into a Spring authority, and applies method-level rules

through `@PreAuthorize`. Requests without a valid token receive HTTP 401, and requests with an insufficient role receive HTTP 403.

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain chain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable()) // stateless API
        .sessionManagement(s -> s.sessionCreationPolicy(
            SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health", "/v3/api-docs/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(j ->
            j.jwtAuthenticationConverter(roleConverter())));
        return http.build();
    }
}
```

Security measures summary

Table 6.2: Security controls and the threats they address

Control	Threat addressed (OWASP category)
bcrypt hashing, login throttling	Cryptographic failures; identification and authentication failures
Short-lived RS256 JWT, refresh-token revocation	Broken authentication; token theft
RBAC at gateway and service	Broken access control
Parameterised queries (JPA, SQLAlchemy, Mongoose) and input validation	Injection
Rate limiting with Redis	Unrestricted resource consumption
Security headers, CORS allow-list, HTTPS at ingress	Security misconfiguration
Dependency and image scanning in CI	Vulnerable and outdated components [35]

6.5 Database Programming: Advanced SQL, Views, Procedures, Triggers and Transactions

External views

Views implement the external level of the three-schema architecture. They hide joins from the services and give each role only the columns it needs.

```
CREATE VIEW nearby_available_donations_v AS
SELECT d.donation_id, d.title, c.name AS category, i.available_qty, d.unit,
       d.expires_at, d.pickup_lat, d.pickup_lng, dn.org_name AS donor
FROM   donations d
JOIN   inventory i      USING (donation_id)
JOIN   food_categories c USING (category_id)
JOIN   donors dn        USING (donor_id)
WHERE  d.status = 'AVAILABLE' AND i.available_qty > 0 AND d.expires_at > now();

-- Donations within 5 km of an NGO, nearest first
SELECT v.*, earth_distance(ll_to_earth(v.pickup_lat, v.pickup_lng),
                          ll_to_earth(17.4948, 78.3996)) / 1000 AS km
FROM   nearby_available_donations_v v
WHERE  earth_box(ll_to_earth(17.4948, 78.3996), 5000)
       @> ll_to_earth(v.pickup_lat, v.pickup_lng)
ORDER BY km, v.expires_at
```

```
LIMIT 20;
```

Joins, subqueries, CTEs, aggregates and window functions

The reporting layer relies on advanced SQL. The following query combines a common table expression with aggregates and three window functions to rank donors by the meals their food produced each month, to show a running total, and to show the change from the previous month.

```
WITH monthly AS (  
    SELECT date_trunc('month', ds.delivered_at) AS month,  
           dn.donor_id, dn.org_name,  
           SUM(ds.servings) AS meals  
    FROM distributions ds  
    JOIN pickups p      USING (pickup_id)  
    JOIN donations d    ON d.donation_id = p.donation_id  
    JOIN donors dn      ON dn.donor_id   = d.donor_id  
    GROUP BY 1, 2, 3  
)  
SELECT month, org_name, meals,  
       RANK() OVER (PARTITION BY month ORDER BY meals DESC) AS rank_in_month,  
       SUM(meals) OVER (PARTITION BY donor_id ORDER BY month) AS cumulative_meals,  
       meals - LAG(meals) OVER (PARTITION BY donor_id ORDER BY month) AS change_vs_prev  
FROM monthly  
ORDER BY month, rank_in_month;
```

Other queries use correlated subqueries and `NOT EXISTS`. For example, the following query finds NGOs that have open requests but have not received any delivery in the last seven days, so that the administrator can prioritise them.

```
SELECT n.ngo_id, n.name, COUNT(r.request_id) AS open_requests  
FROM   ngos n  
JOIN   food_requests r ON r.ngo_id = n.ngo_id AND r.status = 'OPEN'  
WHERE  NOT EXISTS (  
    SELECT 1  
    FROM    pickups p  
    JOIN    food_requests r2 ON r2.request_id = p.request_id  
    WHERE   r2.ngo_id = n.ngo_id  
    AND     p.status = 'DELIVERED'  
    AND     p.picked_at > now() - INTERVAL '7 days')  
GROUP BY n.ngo_id, n.name  
HAVING COUNT(r.request_id) > 0  
ORDER BY open_requests DESC;
```

A materialised view stores daily totals so that dashboards do not scan the whole distribution history. It is refreshed concurrently by a scheduled task, which keeps it readable during the refresh.

```
CREATE MATERIALIZED VIEW daily_impact_mv AS  
SELECT date_trunc('day', delivered_at) AS day,  
       COUNT(*) AS deliveries, SUM(servings) AS meals  
FROM   distributions GROUP BY 1;  
CREATE UNIQUE INDEX ON daily_impact_mv (day);  
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_impact_mv;
```

Stored procedure for reservation

The reservation logic is a procedure so that it runs entirely inside the database as one atomic unit. It locks the inventory row, validates expiry and stock, updates the counters and creates the

pickup. The Spring Boot service calls the same logic through the JPA implementation shown in Section 6.8, and the procedure is retained as a database-level reference and for administrative scripts.

```
CREATE OR REPLACE PROCEDURE reserve_donation(
    p_donation BIGINT, p_request BIGINT, p_qty NUMERIC,
    INOUT p_pickup BIGINT DEFAULT NULL)
LANGUAGE plpgsql AS $$
DECLARE
    v_avail NUMERIC; v_exp TIMESTAMPTZ;
BEGIN
    SELECT i.available_qty, d.expires_at INTO v_avail, v_exp
    FROM inventory i JOIN donations d USING (donation_id)
    WHERE i.donation_id = p_donation
    FOR UPDATE OF i; -- row lock serialises reservations
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Donation % not found', p_donation;
    END IF;
    IF v_exp <= now() THEN
        RAISE EXCEPTION 'Donation % has expired', p_donation;
    END IF;
    IF v_avail < p_qty THEN
        RAISE EXCEPTION 'Insufficient stock: requested %, available %', p_qty, v_avail;
    END IF;
    UPDATE inventory
    SET available_qty = available_qty - p_qty,
        reserved_qty = reserved_qty + p_qty
    WHERE donation_id = p_donation;
    INSERT INTO pickups (donation_id, request_id, reserved_qty)
    VALUES (p_donation, p_request, p_qty) RETURNING pickup_id INTO p_pickup;
    UPDATE donations SET status = 'RESERVED'
    WHERE donation_id = p_donation
    AND (SELECT available_qty FROM inventory WHERE donation_id = p_donation) = 0;
END $$;
```

Triggers

Two triggers protect the integrity of the data regardless of which application changes it. The audit trigger is generic. It receives the name of the primary-key column as an argument and stores the old and new row images as JSON. The status trigger allows only the transitions of the state machine in Figure 5.1.

```
CREATE OR REPLACE FUNCTION trg_audit() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO audit_log (table_name, record_id, action, old_value, new_value)
    VALUES (TG_TABLE_NAME,
        (to_jsonb(COALESCE(NEW, OLD)) ->> TG_ARGV[0])::BIGINT,
        TG_OP,
        CASE WHEN TG_OP <> 'INSERT' THEN to_jsonb(OLD) END,
        CASE WHEN TG_OP <> 'DELETE' THEN to_jsonb(NEW) END);
    RETURN COALESCE(NEW, OLD);
END $$;

CREATE TRIGGER donations_audit AFTER INSERT OR UPDATE OR DELETE ON donations
FOR EACH ROW EXECUTE FUNCTION trg_audit('donation_id');
CREATE TRIGGER pickups_audit AFTER INSERT OR UPDATE OR DELETE ON pickups
FOR EACH ROW EXECUTE FUNCTION trg_audit('pickup_id');

CREATE OR REPLACE FUNCTION trg_donation_status() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.status = OLD.status THEN RETURN NEW; END IF;
    IF (OLD.status, NEW.status) NOT IN (
        ('AVAILABLE', 'RESERVED'), ('AVAILABLE', 'EXPIRED'), ('AVAILABLE', 'CANCELLED'),
        ('RESERVED', 'AVAILABLE'), ('RESERVED', 'PICKUP_SCHEDULED'),
        ('PICKUP_SCHEDULED', 'RESERVED'), ('PICKUP_SCHEDULED', 'COLLECTED'),
        ('COLLECTED', 'DISTRIBUTED')) THEN
```

```

        RAISE EXCEPTION 'Illegal status transition % -> %', OLD.status, NEW.status;
    END IF;
    RETURN NEW;
END $$;

CREATE TRIGGER donations_status BEFORE UPDATE OF status ON donations
FOR EACH ROW EXECUTE FUNCTION trg_donation_status();

```

A third trigger on `ratings` recalculates `volunteers.rating_avg` after each new rating, which is the controlled denormalisation described in Section 5.3.

Transactions, isolation and MVCC

Every business operation that changes more than one row runs in a transaction and therefore obeys the ACID properties: atomicity (all changes or none), consistency (constraints hold before and after), isolation (concurrent transactions do not see each other's partial work) and durability (committed data survives crashes through the write-ahead log) [1][2]. The default isolation level is `READ COMMITTED`. PostgreSQL implements isolation with multi-version concurrency control, so readers see a consistent snapshot and never block writers, while writers to the same row wait for each other. The reservation scenario below was used to verify the behaviour when two NGOs try to reserve the same food at the same moment.

```

-- donation 12 has available_qty = 40
-- Session A
BEGIN;
CALL reserve_donation(12, 5, 30);
-- locks the inventory row, 40 -> 10
COMMIT;

-- Session B
BEGIN;
CALL reserve_donation(12, 6, 30);
-- waits for the row lock
-- resumes: available = 10 < 30
-- ERROR: Insufficient stock
ROLLBACK;

```

Without the row lock, both sessions would read 40 and both would succeed, and the `CHECK` constraint on the inventory would be the last line of defence. With the lock, the second session sees the up-to-date value and fails with a clear message that the service converts into an HTTP 409 response. Long analytical queries use the `REPEATABLE READ` level in read-only mode, so that a report sees one consistent snapshot even while data changes.

6.6 MongoDB Implementation

The User Service defines its collections with Mongoose schemas, which add validation and defaults to MongoDB's flexible model. The role-specific profile is stored as an embedded sub-document, and the geospatial `location` field has a `2dsphere` index that supports "nearest volunteer" queries. Notifications carry a `TTL` index so that old documents are deleted automatically.

```

// user-service/src/models/Notification.js
const notificationSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, index: true, required: true },

```

```

    type:      { type: String, required: true,
                  enum: ['LISTING', 'RESERVATION', 'PICKUP', 'DELIVERY'] },
    title:     { type: String, required: true },
    data:      { type: Schema.Types.Mixed },
    read:      { type: Boolean, default: false },
    createdAt: { type: Date, default: Date.now },
  });
notificationSchema.index({ userId: 1, read: 1, createdAt: -1 });
notificationSchema.index({ createdAt: 1 }, { expireAfterSeconds: 60 * 60 * 24 * 30 });

```

The activity summary shown on each user's dashboard is produced by an aggregation pipeline, which filters, groups and sorts inside the database instead of transferring raw events to the service.

```

const summary = await Activity.aggregate([
  { $match: { userId: new ObjectId(userId), createdAt: { $gte: since } } },
  { $group: { _id: '$type', count: { $sum: 1 }, lastAt: { $max: '$createdAt' } } },
  { $sort: { count: -1 } },
]);

```

Real-time delivery uses Socket.io. Each connection is authenticated with the same JWT, and the socket joins a private room named after the user. Because several replicas of the service may run, the Redis adapter relays events between them, so a notification created on one replica reaches a browser connected to another.

```

io.adapter(createAdapter(pubClient, subClient)); // Redis pub/sub
io.use((socket, next) => {
  try {
    socket.user = jwt.verify(socket.handshake.auth.token, PUBLIC_KEY,
                              { algorithms: ['RS256'] });
    next();
  } catch { next(new Error('unauthorized')); }
});
io.on('connection', (socket) => socket.join(`user:${socket.user.sub}`));

async function notify(userId, payload) {
  const doc = await Notification.create({ userId, ...payload });
  io.to(`user:${userId}`).emit('notification', doc);
}

```

6.7 Vector Search and Retrieval-Augmented Assistant

Semantic matching

When a donation is created, the Matching Service builds a text from its title, description and category, converts it into a 384-dimensional embedding with a Sentence-Transformers model, and stores it. To rank NGO requests, it combines a spatial filter, a category filter and an ordered nearest-neighbour search in one SQL statement, and then calculates the final score in Python. The `<=>` operator is pgvector's cosine distance, so `1 - distance` is the cosine similarity.

```

# matching-service/app/repositories/need_repo.py
RANK_SQL = text("""
  SELECT n.request_id, n.ngo_id,
         1 - (n.embedding <=> CAST(:q AS vector))           AS semantic,
         earth_distance(ll_to_earth(n.latitude, n.longitude),
                        ll_to_earth(:lat, :lng)) / 1000.0    AS km,

```

```

        LEAST(1.0, n.required_qty / :qty)                                AS qty_fit
FROM   need_embeddings n
WHERE  n.status = 'OPEN' AND n.category_id = :cat AND n.needed_by > now()
      AND earth_box(ll_to_earth(:lat, :lng), :radius_m)
      @> ll_to_earth(n.latitude, n.longitude)
ORDER  BY n.embedding <=> CAST(:q AS vector)
LIMIT  20""")

# matching-service/app/services/matching.py
W_SEM, W_DIST, W_QTY = 0.5, 0.3, 0.2

def score(row, radius_km: float) -> float:
    proximity = max(0.0, 1.0 - row.km / radius_km)
    return round(W_SEM * row.semantic + W_DIST * proximity + W_QTY * row.qty_fit, 4)

async def rank_requests(self, req: RankRequest) -> list[MatchOut]:
    qvec = self.embedder.encode(f"{req.title}. {req.description}")
    rows = await self.needs.candidates(qvec, req, radius_m=req.radius_m)
    ranked = sorted(rows, key=lambda r: score(r, req.radius_m / 1000), reverse=True)
    return [MatchOut(request_id=r.request_id, ngo_id=r.ngo_id,
                    score=score(r, req.radius_m / 1000), distance_km=round(r.km, 2))
            for r in ranked[:10]]

```

The HNSW index is tuned at query time with `SET hnsw.ef_search = 40`, which widens the graph search to improve recall at a small cost in latency. The weights were chosen so that a nearby NGO with a moderate semantic match ranks above a distant NGO with a perfect match, because for perishable food, distance matters more than wording.

Food-safety assistant (RAG)

The knowledge base contains short passages from public food-safety and handling guidance, such as safe holding temperatures, time limits for cooked food, and packaging advice. Each passage is embedded and stored in `knowledge_chunks`. When a user asks a question, the service retrieves the most relevant passages with hybrid search, fuses the two ranked lists by reciprocal rank fusion, and gives the top passages to a language model with an instruction to answer only from the supplied context and to cite the sources. If the retrieved context does not contain the answer, the assistant says so instead of guessing.

```

def reciprocal_rank_fusion(*ranked_lists, k: int = 60):
    scores, items = {}, {}
    for ranked in ranked_lists:
        for rank, item in enumerate(ranked, start=1):
            scores[item.chunk_id] = scores.get(item.chunk_id, 0.0) + 1.0 / (k + rank)
            items[item.chunk_id] = item
    order = sorted(scores, key=scores.get, reverse=True)
    return [items[i] for i in order]

class AssistantService:
    async def answer(self, question: str) -> AnswerOut:
        qvec = self.embedder.encode(question)
        vector_hits = await self.chunks.vector_search(qvec, limit=8)
        text_hits = await self.chunks.text_search(question, limit=8)
        top = reciprocal_rank_fusion(vector_hits, text_hits)[:4]
        prompt = build_prompt(question, [c.content for c in top])
        reply = await self.llm.generate(prompt)
        return AnswerOut(answer=reply, sources=sorted({c.source for c in top}))

```

6.8 Distributed Consistency: Saga, Compensation and Cross-Database Integrity

Why not two-phase commit

A reservation touches the Listing database, the Matching database and, indirectly, the User Service. A classic distributed transaction with two-phase commit (2PC) would need a coordinator, would hold locks in every database until all participants vote, and would block if the coordinator failed. It would also couple three technologies that do not share a transaction protocol. The platform therefore uses the **Saga pattern** [11][5]: a long-running operation is split into local transactions, each committed independently, and every step that can be undone has a **compensating action** that reverses its effect if a later step fails. The saga used for "Reserve food and schedule pickup" is shown in Figure 6.1.

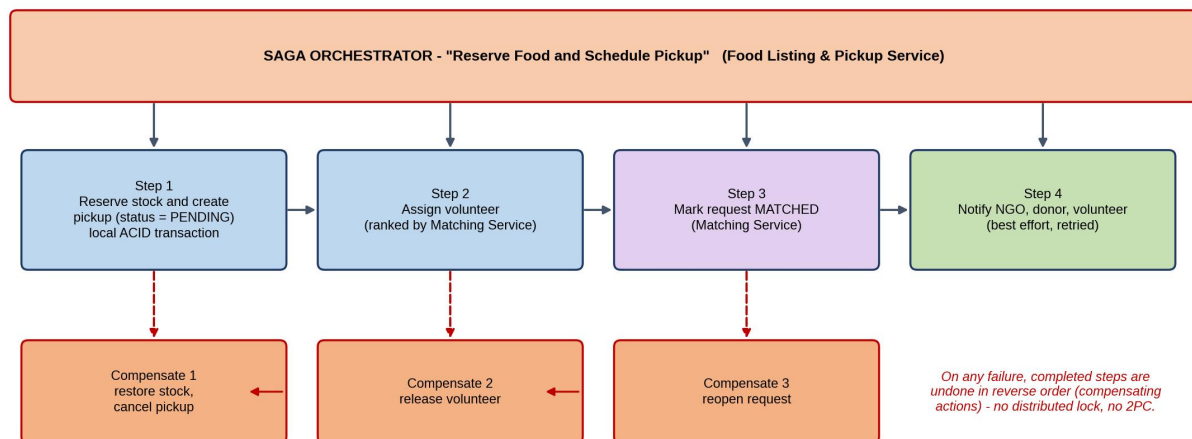


Figure 6.1: Saga orchestration with compensating actions for a reservation

An **orchestrator** class in the Listing Service coordinates the steps. Step 1 is a local ACID transaction. Steps 2 and 3 change state in other places and have compensations. Step 4, the notification, is idempotent and is retried from an outbox table, so its failure does not undo a valid reservation. Compensations are executed in reverse order, each one is itself idempotent, and a compensation that fails is written to a dead-letter table for manual attention.

```
@Service
@RequiredArgsConstructor
@Slf4j
public class ReservationSaga {
    private final ReservationService reservations; // local transaction (row lock)
    private final VolunteerService volunteers;
    private final MatchingClient matching; // REST with timeouts
    private final OutboxService outbox;
    private final DeadLetterService deadLetters;

    public SagaResult run(ReserveCommand cmd) {
        Deque<Runnable> undo = new ArrayDeque<>();
        try {
            Pickup pickup = reservations.reserve(
                cmd.donationId(), cmd.requestId(), cmd.qty());
```

```

        undo.push(() -> reservations.release(pickup.getId())); // C1

        Volunteer v = volunteers.assignBest(pickup, matching);
        undo.push(() -> volunteers.release(v.getId(), pickup.getId())); // C2

        matching.markRequestMatched(cmd.requestId(), cmd.correlationId());
        undo.push(() -> matching.reopenRequest( // C3
            cmd.requestId(), cmd.correlationId()));

        outbox.enqueue(new PickupScheduled(pickup.getId(), v.getId())); // step 4
        return SagaResult.completed(pickup.getId());
    } catch (Exception ex) {
        log.warn("saga failed, compensating: {}", ex.getMessage());
        while (!undo.isEmpty()) {
            try { undo.pop().run(); }
            catch (Exception ce) { deadLetters.record(cmd, ce); }
        }
        return SagaResult.failed(ex.getMessage());
    }
}

```

The local reservation step uses a pessimistic write lock in JPA, which is the object-oriented equivalent of `SELECT ... FOR UPDATE` in the stored procedure:

```

@Repository
public interface InventoryRepository extends JpaRepository<Inventory, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select i from Inventory i where i.donationId = :id")
    Optional<Inventory> lockById(@Param("id") Long id);
}

@Transactional
public Pickup reserve(Long donationId, Long requestId, BigDecimal qty) {
    Inventory inv = inventories.lockById(donationId).orElseThrow(NotFoundException::new);
    Donation d = donations.getReferenceById(donationId);
    if (d.getExpiresAt().isBefore(Instant.now())) throw new ConflictException("Expired");
    if (inv.getAvailableQty().compareTo(qty) < 0)
        throw new InsufficientStockException(qty, inv.getAvailableQty());
    inv.setAvailableQty(inv.getAvailableQty().subtract(qty));
    inv.setReservedQty(inv.getReservedQty().add(qty));
    if (inv.getAvailableQty().signum() == 0) d.setStatus(DonationStatus.RESERVED);
    return pickups.save(Pickup.pending(donationId, requestId, qty));
}

```

Resilience

Every remote call has a connect and read timeout, so that a hung service cannot exhaust the threads of its caller. The gateway's circuit breaker opens after a series of failures and returns HTTP 503 immediately during a cool-down period, which gives the failing service time to recover. Idempotency keys make retried reservation requests safe, because a repeated request returns the stored result. Together, these measures avoid cascading failures.

Consistency between PostgreSQL and MongoDB

Because a relational foreign key cannot span two database engines, the platform uses application-level strategies. First, the registration workflow creates the MongoDB user first and then the relational donor, NGO or volunteer row that stores the user reference. If the second step fails, the first is deleted as a compensation. Second, a nightly reconciliation job compares the

references in both directions and reports orphaned records. Third, the data that must be strongly consistent, such as stock and pickups, is kept in only one place, the relational database. Data that is read often but may be slightly stale, such as a display name shown in a notification, is copied and refreshed by events. This is an application of the CAP trade-off: strong consistency where correctness matters, and eventual consistency where availability and speed matter more [3][12].

6.9 Refactoring the Monolith into Layers and Services

The project did not begin as microservices. The first prototype was a single FastAPI application with all routes, business rules and SQL in a few large files. It was sufficient to prove the data model, but it soon showed the usual problems of a monolith: changes in one feature broke another, tests needed the entire database, and the code of different team members conflicted. The refactoring was carried out in three stages.

1. **Layering.** The routes were moved into routers, the business rules into service classes and the SQL into repository classes, with dependencies injected through `Depends`. The behaviour did not change, but the code became testable in isolation because services could be given fake repositories.
2. **Modularisation.** Related routers, services and repositories were grouped into modules for identity, listings, matching and reports. Modules communicated only through service interfaces, never through each other's tables.
3. **Extraction.** Each module was moved into its own deployable service, and its tables or collections were moved into its own database. The identity module became the Node.js service and the listing module became the Spring Boot service, which allowed the team to apply the two other frameworks of the course. The FastAPI application that remained became the gateway and the Matching service.

The stage-wise approach kept the system working at every step. It also confirmed a practical lesson: clear layering inside a monolith is the best preparation for splitting it later, because the boundaries that are needed for services already exist as boundaries between modules.

CHAPTER - 7 FEATURES

7.1 List of Main Features

The platform provides the features listed in Table 7.1. Each feature is described in Section 7.2 from the point of view of the user and from the technical point of view.

Table 7.1: Main features of the platform

No.	Feature	Primary users	Main components
F1	Role-based registration and login	All	User Service, MongoDB, JWT
F2	Surplus food listing and status tracking	Donors	Listing Service, PostgreSQL
F3	Nearby donation discovery	NGOs, volunteers	PostgreSQL GiST index, gateway cache
F4	Food request management	NGOs	Listing Service, Matching read model
F5	Smart matching of donations and requests	NGOs, administrators	Matching Service, pgvector
F6	Safe reservation of food	NGOs	Row locks, Saga orchestrator
F7	Volunteer assignment and pickup workflow	Volunteers, NGOs	Listing Service, Matching Service
F8	Distribution recording and impact tracking	Volunteers, NGOs	distributions table, materialised views
F9	Real-time notifications and activity feed	All	Socket.io, Redis, MongoDB
F10	Reports and analytics dashboard	Administrators, NGOs	Advanced SQL, Recharts
F11	Food-safety assistant	All	Hybrid search, RAG
F12	Audit trail and traceability	Administrators	Triggers, audit_log
F13	Ratings and feedback	Donors, NGOs	ratings table, trigger
F14	Monitoring and alerting	Operators	OpenTelemetry, Prometheus, Grafana

7.2 How Each Feature Works

F1. Role-based registration and login

User perspective. A new user chooses a role (donor, NGO or volunteer), enters the details relevant to that role and logs in. An NGO account remains restricted until an administrator verifies its registration number. After login, the user sees only the dashboard and actions that belong to the role.

Technical description. The User Service validates the input, hashes the password with bcrypt and stores the user document in MongoDB. It then calls the Listing Service to create the matching donor, NGO or volunteer row, and stores the returned identifier in the document.

Login returns a short-lived RS256 access token and a revocable refresh token. The gateway and the Spring Boot service verify the token on every request and check the role claim.

F2. Surplus food listing and status tracking

User perspective. A donor fills a short form with the food category, quantity and unit, preparation time, safe-consumption deadline and pickup location. The form suggests a default deadline from the category's shelf life. After submission, the donor sees a card whose status badge changes from AVAILABLE to RESERVED, PICKUP_SCHEDULED, COLLECTED and DISTRIBUTED as the food moves along.

Technical description. The Listing Service inserts the donation and its inventory row in one transaction. CHECK constraints reject non-positive quantities and deadlines earlier than the preparation time. The status trigger allows only legal transitions, the audit trigger records each change and the scheduler expires overdue listings. The gateway forwards the change events to the User Service, which stores them in the donor's activity feed.

F3. Nearby donation discovery

User perspective. An NGO coordinator or volunteer opens a map or list of available food near a chosen location. They can filter by category, minimum quantity and how much time remains. Results are sorted by distance and urgency.

Technical description. The query reads the view `nearby_available_donations_v` and uses `earth_box` with the GiST index to restrict rows to the requested radius before computing exact distances. The partial index on available donations keeps the scan small. Frequently used searches are cached in Redis for a few seconds, and the cache is invalidated when a listing changes.

F4. Food request management

User perspective. An NGO states what it needs, for example "cooked meals for 60 people by 8 pm", and can update or cancel the request. The request stays open until it is matched, fulfilled or withdrawn.

Technical description. The Listing Service stores the request and sends its category, quantity, deadline, location and text to the Matching Service, which embeds the text and stores it in the read model `need_embeddings`. When the request changes state, the read model is updated so that stale requests are never proposed.

F5. Smart matching of donations and requests

User perspective. When food is listed, the platform proposes the NGOs whose requests fit best. NGO coordinators see a ranked list of suitable donations, and administrators can see why a match was proposed, through the three sub-scores for meaning, distance and quantity.

Technical description. The Matching Service filters open requests by category, deadline and radius, orders them by cosine distance between embeddings and combines the semantic similarity, proximity and quantity fit into a weighted score (Section 6.7). The result is returned as a ranked list with the sub-scores. Weights are configuration values and can be tuned without redeploying the service.

F6. Safe reservation of food

User perspective. An NGO selects a donation, enters the quantity it can collect and confirms. If another NGO has taken part of the food in the meantime, the NGO sees a clear message with the remaining quantity instead of an unexplained failure.

Technical description. The reservation runs as the Saga of Section 6.8. A row lock on the inventory serialises concurrent requests, CHECK constraints protect the stock invariant, an idempotency key protects against double clicks and retries, and compensating actions restore the previous state if a later step fails.

F7. Volunteer assignment and pickup workflow

User perspective. After a reservation, the nearest available volunteer of the NGO receives a pickup task with the donor's address, contact and deadline, and a link that opens navigation. The volunteer confirms collection at the donor's location and delivery at the destination.

Technical description. The Listing Service queries the NGO's available volunteers from its own database and sends their locations to the Matching Service, which returns them ranked by distance and rating. The best candidate is assigned and marked busy. Collection and delivery are POST operations that move the pickup and donation states and are recorded by the audit trigger with timestamps.

F8. Distribution recording and impact tracking

User perspective. When food reaches beneficiaries, the volunteer or NGO records the beneficiary group and the number of servings. The donor receives a confirmation, and dashboards update with meals served.

Technical description. A row is inserted into `distributions`, and the inventory moves the quantity from reserved to distributed in the same transaction. Daily totals are aggregated in the materialised view `daily_impact_mv`, which is the source of the impact charts.

F9. Real-time notifications and activity feed

User perspective. Users receive an instant alert when something relevant happens: a new donation near an NGO, a reservation confirmed for a donor, a task assigned to a volunteer or a delivery completed. The bell icon shows unread items, and a feed lists past activity.

Technical description. Events are stored as notification documents in MongoDB and pushed through Socket.io to the private room of the receiving user. The Redis adapter shares events between service replicas. Documents expire after 30 days through a TTL index. If the user is offline, the notification is available on the next login.

F10. Reports and analytics dashboard

User perspective. Administrators and NGO managers see charts of meals distributed, food saved by category, response times, expiry rate and top donors, and can filter by period and area.

Technical description. The Matching and Insight Service runs SQL that combines joins, aggregates, common table expressions and window functions (Section 6.5) and returns compact JSON for the charts. Results for large periods use the materialised views, so that the dashboards remain responsive.

F11. Food-safety assistant

User perspective. A donor or volunteer can ask questions such as "How long can cooked rice be kept before donation?" and receives a short answer that lists its sources. If the knowledge base does not cover the question, the assistant says so.

Technical description. The question is embedded, hybrid search combines full-text and vector retrieval, the fused passages are supplied as context to a language model and the answer is returned with the source names (Section 6.7). Answers are advisory and never replace official guidance.

F12. Audit trail and traceability

User perspective. An administrator can open any donation and see its complete history: who created it, who reserved it, who collected it, when, and to which beneficiary group it was delivered.

Technical description. Row-level triggers write the old and new values of donations and pickups to `audit_log` as JSON. The composite index on table name, record identifier and time makes retrieval fast. The log is append-only, and the application role has no permission to update or delete its rows.

F13. Ratings and feedback

User perspective. After delivery, the NGO and the donor can rate the experience from one to five stars and leave a comment. Volunteers see their average rating, and administrators see recurring issues.

Technical description. Ratings are stored one per distribution through a unique constraint. A trigger recalculates the volunteer's average, which is the denormalised value discussed in Section 5.3.

F14. Monitoring and alerting

User perspective. Operators and evaluators can open a Grafana dashboard that shows the health of every service and the business activity in real time.

Technical description. Services export OpenTelemetry traces and metrics to a collector, Prometheus scrapes and stores the metrics, and Grafana visualises them. Alert rules raise a notification when the error rate, latency or number of expiring listings crosses a threshold (Chapter 9).

CHAPTER - 8 TESTING

Testing was planned as a pyramid. Many fast unit tests check individual functions and services, fewer integration tests check the cooperation of services with real databases, and a small number of end-to-end and load tests check complete workflows and performance. Every level runs automatically in the continuous integration pipeline, and a failing test blocks the merge.

Table 8.1: Testing levels and tools

Level	Scope	Tools
Unit	Functions, services, validators, scoring logic	pytest (Python), Jest (Node.js), JUnit 5 with Mockito (Java)
Component / API	One service with a real database	pytest with httpx TestClient, Supertest, Spring MockMvc
Integration	Several services and databases together	Docker Compose test profile, Testcontainers, Postman/Newman
End-to-end	Complete donor to beneficiary journey	Newman collection run against the Compose stack
Load and resilience	Throughput, latency, failure behaviour	k6 and Locust
Security	Dependencies, images, static analysis	OWASP Dependency-Check, Trivy, Bandit, npm audit

8.1 Postman Testing (Frontend/Backend)

Postman was used from the first day of API development. Because the API contract was designed first (Section 5.8), the team created a Postman collection with one folder per resource and one request per endpoint. Requests use collection variables for the base URL and environment variables for tokens, so the same collection runs against a developer machine, the Compose stack and the staging cluster. Test scripts written in JavaScript check the status code, the response schema, the presence of the correlation identifier and business rules such as "the reserved quantity equals the requested quantity".

A login request at the start of the collection stores the access token in an environment variable, and later requests use it automatically. Separate folders authenticate as donor, NGO, volunteer and administrator, so that role-based access control is tested from both sides: an allowed call must succeed and a forbidden call must return HTTP 403. The same collection is executed automatically in the pipeline with Newman, the command-line runner of Postman.

```
// Postman test script for POST /api/v1/donations/{id}/reservations
pm.test("status is 201", () => pm.response.to.have.status(201));
pm.test("response has pickup id and PENDING or SCHEDULED status", () => {
  const body = pm.response.json();
  pm.expect(body.pickupId).to.be.a("number");
  pm.expect(["PENDING", "SCHEDULED"]).to.include(body.status);
});
pm.test("correlation id is returned", () => {
```

```
pm.expect(pm.response.headers.get("X-Correlation-ID")).to.be.ok;
});
pm.environment.set("pickupId", pm.response.json().pickupId);
```

For the front end, the browser's developer tools and the Postman console were used to compare the payloads sent by the React application with the contract. Contract mismatches, such as a field named `expiry` in the form and `expiresAt` in the API, were detected this way and corrected before integration.

8.2 Integration Testing

Integration tests check that the pieces work together when they use real infrastructure rather than mocks. Each backend service has tests that start throw-away PostgreSQL, MongoDB and Redis containers, apply the real schema scripts, and call the service through HTTP. This approach catches problems that unit tests cannot, such as a constraint that rejects valid data, an index that is missing or a query that is correct in memory but slow on real tables.

Service-level examples

The FastAPI Matching Service tests use the `TestClient` and override the authentication dependency, which is possible because of FastAPI's dependency injection. The Spring Boot tests use Testcontainers so that the same PostgreSQL 16 image as in production is used.

```
# matching-service/tests/test_rank.py
import pytest
from fastapi.testclient import TestClient
from app.main import app
from app.deps import current_user

app.dependency_overrides[current_user] = lambda: {"sub": "u1", "role": "NGO"}
client = TestClient(app)

def test_rank_prefers_nearby_ngo(seed_needs):
    r = client.post("/matches/rank", json={
        "title": "Vegetable biryani", "description": "30 plates, cooked today",
        "category_id": 1, "quantity": 30, "lat": 17.49, "lng": 78.40, "radius_m": 8000})
    assert r.status_code == 200
    scores = [m["score"] for m in r.json()]
    assert scores == sorted(scores, reverse=True)
    assert r.json()[0]["distance_km"] <= r.json()[-1]["distance_km"] + 3
```

```
// listing-service/src/test/java/.../ReservationConcurrencyIT.java
@SpringBootTest
@Testcontainers
class ReservationConcurrencyIT {
    @Container static PostgreSQLContainer<?> pg = new
    PostgreSQLContainer<>("postgres:16");

    @Autowired ReservationService service;
    @Autowired InventoryRepository inventories;

    @Test
    void twoNgosCannotReserveMoreThanAvailable() throws Exception {
        Long id = TestData.donationWithStock(new BigDecimal("40"));
        ExecutorService pool = Executors.newFixedThreadPool(2);
        Callable<Boolean> task = () -> {
```

```

        try { service.reserve(id, null, new BigDecimal("30")); return true; }
        catch (InsufficientStockException e) { return false; }
    };
    List<Future<Boolean>> results = pool.invokeAll(List.of(task, task));
    long successes = results.stream().filter(f -> get(f)).count();
    assertEquals(1, successes);
    assertEquals(0, new BigDecimal("10").compareTo(
        inventories.findById(id).orElseThrow().getAvailableQty()));
}
}

```

The concurrency test is the most valuable integration test in the project, because it proves the central promise of the system: food is never promised twice.

Cross-service scenarios

The end-to-end scenarios run through the gateway with the whole Compose stack. They cover the complete journey (register, list, match, reserve, collect, distribute), the failure of a downstream service during a reservation (the Matching Service container is stopped in the test, and the test verifies that stock and request state return to their previous values), expiry handling and notification delivery over a Socket.io connection.

8.3 Tools Used for Testing

Table 8.2: Testing tools and their purpose

Tool	Purpose
pytest and pytest-cov	Python unit and component tests; coverage measurement [40]
httpx / FastAPI TestClient	In-process API calls to FastAPI apps
Jest and Supertest	Node.js unit and HTTP tests [41]
JUnit 5, Mockito, Spring MockMvc	Java unit and web-layer tests
Testcontainers	Disposable PostgreSQL, MongoDB and Redis containers for integration tests
Postman and Newman	Manual and automated API tests [47]
k6 and Locust	Load, stress and soak testing [45]
Trivy	Vulnerability scan of container images
OWASP Dependency-Check, npm audit, Bandit	Vulnerable dependencies and insecure code patterns [35]
EXPLAIN (ANALYZE, BUFFERS)	Query plan inspection for SQL tuning
Grafana dashboards	Observation of latency and errors during load tests

Load testing

Load tests were written with k6 to check the search and reservation endpoints under increasing numbers of virtual users. The script below ramps the load up, holds it and ramps it down, and defines thresholds. If a threshold is violated, k6 exits with a failure code, so the same script can be used as a pipeline gate.

```

import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [ { duration: '1m', target: 50 }, { duration: '3m', target: 50 },
    { duration: '30s', target: 0 } ],
  thresholds: { http_req_duration: ['p(95)<300'], http_req_failed: ['rate<0.01'] },
};

export default function () {
  const res = http.get(`${__ENV.BASE}/api/v1/donations?lat=17.49&lng=78.40&radius=5000`,
    { headers: { Authorization: `Bearer ${__ENV.TOKEN}` } });
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);
}

```

8.4 Test Cases and Results

Functional test cases

Table 8.3 lists the principal functional test cases. Each case has an identifier that is also used in the automated test names, so that a failing test can be traced to the requirement it protects.

Table 8.3: Functional test cases

ID	Feature	Scenario and input	Expected result	Level
TC01	F1	Register a donor with a valid e-mail and an 8-character password	201 Created; password stored only as a bcrypt hash	API
TC02	F1	Register with an e-mail that already exists	409 Conflict with code EMAIL_EXISTS	API
TC03	F1	Register with role ADMIN	422 Unprocessable Entity; role not allowed	API
TC04	F1	Login with a wrong password	401 Unauthorized; no token issued	API
TC05	F1	Call a protected endpoint with an expired token	401 Unauthorized	API
TC06	F2	Donor creates a listing with quantity 0	422; CHECK and Bean Validation reject it	API, DB
TC07	F2	Donor creates a listing whose expiry is before preparation time	422; constraint violation reported	API, DB
TC08	F2	Donor creates a valid listing	201; donation and inventory rows created together	Integration
TC09	F2	NGO tries to create a listing	403 Forbidden	API
TC10	F3	NGO searches within 5 km	Only AVAILABLE, unexpired listings in the radius, nearest first	Integration
TC11	F6	NGO reserves 30 of 40 units	201; available 10, reserved 30, pickup PENDING	Integration
TC12	F6	Two NGOs reserve 30 units each of 40 at once	Exactly one succeeds; the other receives 409 INSUFFICIENT_STOCK	Concurrency
TC13	F6	Same reservation sent twice with one Idempotency-Key	One pickup created; second call returns the stored result	API
TC14	F6	Reservation while the Matching Service is down	Saga compensates; stock and request state restored; 503 returned	End-to-end

ID	Feature	Scenario and input	Expected result	Level
TC15	F2	Listing passes its expiry time	Scheduler sets status EXPIRED; listing disappears from search	Integration
TC16	F2	Direct SQL update from DISTRIBUTED to AVAILABLE	Trigger raises Illegal status transition	DB
TC17	F7	Volunteer confirms collection of another volunteer's pickup	403 Forbidden	API
TC18	F8	Record distribution of more servings than delivered quantity allows	422; inventory invariant preserved	API, DB
TC19	F9	Reservation confirmed	Donor receives a Socket.io notification; document stored in MongoDB	End-to-end
TC20	F12	Update a donation	New row in audit_log with old and new JSON values	DB
TC21	F5	Rank requests for a donation	Scores are sorted in descending order and within 0 to 1	Unit
TC22	F11	Ask a question that the knowledge base cannot answer	Assistant states that it has no reliable source	Component
TC23	Security	101 requests from one address within a minute	The 101st request receives 429 Too Many Requests	API
TC24	Security	SQL text in a search parameter	Treated as data; no error, no injection	Security

Acceptance criteria and recording of results

Each pipeline run stores the test reports, the coverage report, the Newman summary and the k6 summary as build artefacts, so that the result of every test case is available for the exact commit that was tested. The acceptance criteria that the pipeline enforces are listed in Table 8.4. A build that does not satisfy them is rejected.

Table 8.4: Acceptance criteria enforced by the pipeline

Criterion	Threshold	Enforced by
All unit, component and integration tests	100 percent pass	pytest, Jest, JUnit
Line coverage of service and repository layers	At least 80 percent	pytest-cov, JaCoCo, Jest coverage
Postman / Newman collection	All assertions pass	Newman in CI
Search endpoint latency under 50 virtual users	95th percentile below 300 ms	k6 threshold
Error rate under load	Below 1 percent	k6 threshold
Critical and high vulnerabilities in images	None allowed	Trivy exit code
Vulnerable dependencies	None with high severity	OWASP Dependency-Check, npm audit

The results obtained for the team's own runs, such as measured coverage percentages, latency percentiles and the query plans before and after tuning, should be attached from the build artefacts and Grafana dashboards of the final run.

CHAPTER - 9 DEPLOYMENT

The deployment strategy has one goal: the same versioned container images must run identically on a laptop, in a test environment and in a cloud cluster. This chapter explains the steps to deploy, the configuration of each environment, and the hosting of the front end, the back-end services and the supporting infrastructure.

9.1 Steps to Deploy

Step 1: Prepare the source and configuration

The repository is cloned, and the template file `.env.example` is copied to `.env`. The values that must be set are the database passwords, the RSA key pair used to sign and verify tokens, the Redis URL and the URLs of the internal services. The private key is mounted only into the User Service, and the public key is given to every service that verifies tokens.

Step 2: Build container images

Every service has a multi-stage Dockerfile. The first stage compiles or installs dependencies with the full toolchain, and the last stage copies only the runtime files into a small base image and runs as a non-root user. This keeps images small and reduces the attack surface.

```
# gateway/Dockerfile (FastAPI)
FROM python:3.11-slim AS build
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim
RUN useradd --create-home appuser
WORKDIR /app
COPY --from=build /install /usr/local
COPY app ./app
USER appuser
EXPOSE 8000
HEALTHCHECK CMD python -c "import urllib.request as u;
u.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

# listing-service/Dockerfile (Spring Boot)
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:17-jre
RUN useradd --create-home appuser
COPY --from=build /src/target/listing-service.jar /app/app.jar
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Step 3: Start the stack with Docker Compose

Docker Compose describes the whole system in one file. Services communicate on a private network by service name, the databases keep their data in named volumes, and health checks make dependent services wait until PostgreSQL and MongoDB accept connections. Only the front end, the gateway and Grafana publish ports to the host.

```
services:
  postgres:
    image: pgvector/pgvector:pg16
    environment: { POSTGRES_PASSWORD: ${PG_PASSWORD} }
    volumes: [ "pgdata:/var/lib/postgresql/data", "./db/init:/docker-entrypoint-
initdb.d" ]
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U postgres"], interval: 5s, retries:
10 }
    networks: [ backend ]
  mongo:
    image: mongo:7
    volumes: [ "mongodata:/data/db" ]
    networks: [ backend ]
  redis:
    image: redis:7-alpine
    networks: [ backend ]
  user-service:
    build: ./user-service
    environment: { MONGO_URL: mongodb://mongo:27017/users, REDIS_URL: redis://redis:6379 }
    depends_on: [ mongo, redis ]
    networks: [ backend ]
  listing-service:
    build: ./listing-service
    environment: { SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/surplus_db }
    depends_on: { postgres: { condition: service_healthy } }
    networks: [ backend ]
  matching-service:
    build: ./matching-service
    environment: { DATABASE_URL:
postgresql+psycpg://postgres:${PG_PASSWORD}@postgres/matching_db }
    depends_on: { postgres: { condition: service_healthy } }
    networks: [ backend ]
  gateway:
    build: ./gateway
    ports: [ "8000:8000" ]
    environment: { REDIS_URL: redis://redis:6379 }
    depends_on: [ user-service, listing-service, matching-service ]
    networks: [ backend, edge ]
  frontend:
    build: ./frontend
    ports: [ "80:80" ]
    networks: [ edge ]
volumes: { pgdata: {}, mongodata: {} }
networks: { backend: {}, edge: {} }
```

The stack is started with `docker compose up --build -d`, and `docker compose ps` shows the health of each container. The SQL scripts in `db/init` create the two databases, the extensions, the schema, the views, procedures and triggers, and load the reference data such as food categories.

Step 4: Deploy to Kubernetes

For the cloud target, the images are pushed to the registry and the manifests in `deploy/k8s` are applied. Each service has a Deployment with two replicas, resource requests and limits, and

readiness and liveness probes that call the health endpoint. A Service gives every Deployment a stable internal address, and an Ingress exposes the front end and the gateway.

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: listing-service, namespace: food-surplus }
spec:
  replicas: 2
  selector: { matchLabels: { app: listing-service } }
  template:
    metadata: { labels: { app: listing-service } }
    spec:
      containers:
        - name: listing-service
          image: ghcr.io/food-surplus/listing-service:1.0.0
          ports: [ { containerPort: 8080 } ]
          envFrom: [ { configMapRef: { name: listing-config } },
                    { secretRef: { name: listing-secrets } } ]
          resources:
            requests: { cpu: 250m, memory: 512Mi }
            limits: { cpu: 500m, memory: 1Gi }
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
---
apiVersion: v1
kind: Service
metadata: { name: listing-service, namespace: food-surplus }
spec:
  selector: { app: listing-service }
  ports: [ { port: 80, targetPort: 8080 } ]
```

The commands `kubectl apply -f deploy/k8s/`, `kubectl rollout status deployment/listing-service` and `kubectl get pods -n food-surplus` complete the deployment and verify it. If a new version misbehaves, `kubectl rollout undo` returns to the previous revision.

Step 5: Verify the deployment

A smoke test calls the health endpoint of the gateway, registers a test donor through the API, creates a listing and confirms that it appears in search results. The Grafana dashboard is then opened to confirm that all services report metrics. The same smoke test runs at the end of the pipeline after each deployment.

9.2 Environment Configuration

Configuration follows the twelve-factor principle that anything that changes between environments belongs in the environment, not in the code. Table 9.1 lists the main variables. Secrets are supplied through Docker or Kubernetes secrets and GitHub encrypted secrets, and they never appear in the repository or in the images.

Table 9.1: Principal configuration variables

Variable	Used by	Purpose
----------	---------	---------

Variable	Used by	Purpose
PG_PASSWORD	PostgreSQL, Listing and Matching services	Database credential
MONGO_URL	User Service	MongoDB connection string
REDIS_URL	Gateway, User Service	Cache, rate limiting and Socket.io adapter
JWT_PRIVATE_KEY	User Service only	Signs access tokens
JWT_PUBLIC_KEY	Gateway, Listing Service, Matching Service	Verifies tokens
SERVICE_URLS	Gateway	Base URLs of the internal services
RATE_LIMIT_PER_MIN	Gateway	Request limit per client and minute
EMBEDDING_MODEL	Matching Service	Name of the embedding model
OTEL_EXPORTER_OTLP_ENDPOINT	All services	Address of the OpenTelemetry Collector
CORS_ALLOWED_ORIGINS	Gateway	Origins allowed to call the API

Environments

Table 9.2: Environments and their differences

Environment	Purpose	Notable settings
Local	Development on a laptop	Compose stack, seed data, verbose logging, SMS disabled
Test / CI	Automated pipeline	Disposable containers, fixed seed, no external calls
Staging	Demonstration and final checks	Kubernetes, realistic data volume, TLS, monitoring on
Production	Live use	Two or more replicas, managed backups, alerts on, strict CORS

Database inventory

Because the system is polyglot, an inventory of data stores is maintained so that operators know what exists, who owns it and how it is protected.

Table 9.3: Database inventory

Store	Owner	Contents	Backup and protection
PostgreSQL <code>surplus_db</code>	Listing Service	Donors, NGOs, volunteers, beneficiaries, donations, inventory, requests, pickups, distributions, ratings, audit log	Daily logical dump with <code>pg_dump</code> and write-ahead-log archiving; least-privilege application role
PostgreSQL <code>matching_db</code>	Matching Service	Embeddings, request read model, knowledge chunks	Rebuildable from source data; weekly dump
MongoDB <code>users</code>	User Service	Users, refresh tokens, activities, notifications	Daily mongodump; TTL removes old notifications

Store	Owner	Contents	Backup and protection
Redis	Gateway, User Service	Counters, cache, pub/sub channels	Not persistent by design; contents can be lost safely

9.3 Hosting of Frontend and Backend

Front end

The React application is built into static files by `vite build`. A small NGINX image serves the files, applies gzip compression and long cache lifetimes for hashed assets, and falls back to `index.html` for client-side routes. The same image runs in Compose and in Kubernetes. Alternatively, the static build can be hosted on any static hosting service or CDN, with the API base URL provided at build time.

Back end

The four back-end services and the databases run inside the cluster. On a small budget, the platform can be hosted on a single virtual machine with Docker Compose and an NGINX reverse proxy that terminates TLS with a free certificate. For a production deployment the team recommends a managed Kubernetes service with managed PostgreSQL and MongoDB, so that backups, patching and failover are handled by the provider.

Continuous integration and delivery

The pipeline is defined in `.github/workflows/ci.yml`. It builds and tests each service in a matrix, scans the images and dependencies, and pushes images tagged with the commit hash when the branch is `main`.

```
name: ci
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  test:
    runs-on: ubuntu-latest
    strategy: { matrix: { service: [gateway, matching-service, user-service, listing-service] } }
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: make -C ${ matrix.service } test
  security:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build gateway image
        run: docker build -t gateway:${{ github.sha }} gateway
      - name: Trivy image scan
        uses: aquasecurity/trivy-action@0.24.0
        with:
          image-ref: "gateway:${{ github.sha }}"
          severity: "CRITICAL,HIGH"
          exit-code: "1"
      - name: OWASP dependency check
        run: make dependency-check
  publish:
```

```
needs: security
if: github.ref == 'refs/heads/main'
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: docker/login-action@v3
    with:
      registry: ghcr.io
      username: "${{ github.actor }}"
      password: "${{ secrets.GITHUB_TOKEN }}"
  - run: make publish TAG=${{ github.sha }}
```

Monitoring configuration

Prometheus scrapes the metrics endpoint of the collector and the services, and Grafana reads Prometheus as a data source. The provisioned dashboards show request rate, error rate and the 50th, 95th and 99th percentile latency for each service, database connection pool usage, JVM and Node.js memory, the number of active listings, the number of expiring listings and meals distributed per day. Alert rules notify the team when the 5xx error rate exceeds 2 percent for five minutes, when the 95th percentile latency of the gateway exceeds 500 ms, or when more than a set number of listings are about to expire without a reservation.

```
groups:
  - name: food-surplus
    rules:
      - alert: HighErrorRate
        expr: >
          sum(rate(http_server_requests_total{status=~"5.."}[5m]))
          / sum(rate(http_server_requests_total[5m])) > 0.02
        for: 5m
        labels: { severity: critical }
        annotations: { summary: "5xx error rate above 2 percent" }
```

Assembled request flow

When all parts are running, a request follows the flow of Figure 4.3. The browser calls the Ingress, which sends the request to the gateway. The gateway verifies the token, checks the role and the rate limit and forwards the request with a correlation identifier to the responsible service. The service reads or writes its own database, calls other services when needed and returns a response that the gateway relays to the browser. Because the correlation identifier appears in every log line and trace, an operator can follow a single request across four services and three data stores.

CHAPTER - 10 CHALLENGES & LIMITATIONS

10.1 Technical Challenges Faced

Building a polyglot, multi-framework, microservice system inside a single academic project exposed the team to problems that a single-framework assignment would not raise. The main challenges and how they were resolved are listed in Table 10.1.

Table 10.1: Technical challenges and their resolution

Challenge	Description	Resolution
Preventing double reservation	Two NGOs could reserve the same food at the same instant	Row-level locking with <code>SELECT ... FOR UPDATE</code> inside a stored procedure and a JPA pessimistic lock, backed by a CHECK constraint as a last line of defence (Section 6.5)
Consistency across databases	A foreign key cannot span PostgreSQL and MongoDB	Application-level linking with compensating creation, plus a nightly reconciliation job (Section 6.8)
Coordinating three frameworks	FastAPI, Express and Spring Boot have different conventions for validation, errors and configuration	A shared error envelope and OpenAPI contract agreed before implementation; consistent environment-variable naming
Distributed failure handling	A reservation touches three services; any one can fail mid-operation	Saga pattern with compensating actions and an idempotency key, rather than a two-phase commit
Real-time delivery across replicas	Socket.io connections are pinned to one replica of the User Service	Redis adapter for Socket.io so that events reach every replica
Tuning vector search	Early HNSW settings gave fast but sometimes poor matches for edge-case categories	Increased <code>ef_search</code> for queries and re-benchmarked recall against a hand-labelled test set
Learning curve of Spring Security and JPA	The team's first exposure to Spring's declarative security and lazy loading	Guided study sessions, smaller katas before the real module, careful review of transaction boundaries
Keeping Docker images small and secure	Initial images included build tools and were large	Multi-stage builds and Trivy scanning caught unused layers and known vulnerabilities
Query performance on the search endpoint	The naive query scanned all donations before geo-filtering	Partial and GiST indexes plus <code>EXPLAIN ANALYZE</code> -guided tuning (Section 5.4)
CI pipeline duration	Running every service's tests sequentially was slow	A matrix strategy in GitHub Actions runs the services in parallel

10.2 Team Coordination Challenges

Coordinating three team members across three back-end frameworks required deliberate process, because the parts had to interoperate correctly even though they were built somewhat independently.

- **Divergent local environments.** Different operating systems produced small differences in line endings and file permissions. This was solved with a `.gitattributes` file and Docker-based development, so that the same containers ran regardless of the host.
- **API contract drift.** Before the OpenAPI contract was agreed as the source of truth, the front end and a service occasionally disagreed on a field name. Weekly contract reviews and generated client types reduced this to zero after the first month.
- **Merge conflicts in shared SQL scripts.** Two members editing the same schema file at the same time caused conflicts. The team split the schema into per-module files and adopted a rule that schema changes go through a short review before merging.
- **Uneven familiarity with frameworks.** Members were more comfortable with Python than with Java or Node.js. Pair programming sessions and short internal walkthroughs of Spring Boot and Express brought everyone to a working level.
- **Scheduling around the review calendar.** Splitting work into three services meant that a feature sometimes needed changes in two services at once. The team scheduled cross-service features so that the dependent services were built in the same sprint.

10.3 Current Limitations of the System

Despite meeting its objectives, the current implementation has limitations that are acknowledged honestly rather than hidden.

- **No route optimisation.** Volunteer navigation uses straight-line distance and an external map link; it does not compute optimised multi-stop routes or account for live traffic.
- **Single-region deployment.** The reference deployment runs in one Kubernetes cluster and one region. There is no cross-region replication or disaster-recovery failover yet.
- **Manual NGO verification.** An administrator currently verifies NGO registration numbers by inspection; there is no integration with a government registry API.
- **Text-only donation descriptions.** Donors describe food in text; there is no photo upload or computer-vision check of freshness or quantity.
- **English-only assistant and interface.** The food-safety assistant and the interface currently support English only, which limits reach in multilingual communities.

- **Simplified consistency between PostgreSQL and MongoDB.** The reconciliation job runs nightly, so a broken link between a user and a donor/NGO/volunteer row can persist for up to a day before being reported.
- **Limited offline support.** The progressive web application caches its shell but does not queue actions performed while offline, such as confirming a pickup with no network coverage.
- **No payment or logistics-cost handling.** The scope intentionally excludes monetary transactions, so donors who wish to cover transport costs cannot do so through the platform.
- **Fixed matching weights.** The weights that combine semantic similarity, distance and quantity fit are configuration values tuned manually; they are not learned from outcome data yet.
- **Load testing at moderate scale.** Performance was verified up to the levels described in Chapter 8; behaviour at a much larger scale, for example thousands of concurrent NGOs in one city, was not measured.

These limitations define the starting point for the enhancements discussed in Chapter 11.

CHAPTER - 11 FUTURE ENHANCEMENTS

11.1 Planned Features

The following features were identified as valuable extensions but were kept out of the current scope so that the core redistribution workflow could be completed and tested thoroughly.

Table 11.1: Planned features and the problem they address

Planned feature	Addresses	Sketch of the approach
Route optimisation for volunteers	Section 10.3, no route optimisation	A routing service using a vehicle-routing heuristic, consuming pickup and delivery points and returning an ordered stop list
Photo verification of donations	Text-only descriptions	Image upload with a lightweight model that flags obviously spoiled food for manual review, not automatic rejection
Government NGO-registry integration	Manual NGO verification	A verification adapter that calls the registry API where available, falling back to manual review
Multi-language interface and assistant	English-only support	Interface strings in a translation catalogue; the RAG assistant's knowledge base translated and retrieved per requested language
Offline action queue	Limited offline support	A service worker that queues pickup and delivery confirmations locally and replays them with idempotency keys when connectivity returns
Learned matching weights	Fixed matching weights	Logging of match outcomes (accepted or rejected) as training data for a periodically retrained scoring model, with the current linear score kept as a safe fallback
Donor cost-offset option	No payment handling	An optional, clearly separated module for donors to cover a volunteer's transport cost, kept isolated from the core redistribution data
Cross-region deployment	Single-region deployment	A second Kubernetes cluster with asynchronous PostgreSQL replication and a global load balancer

11.2 Possible Scalability Improvements

- **Read replicas and connection pooling.** As search traffic grows, PostgreSQL read replicas can serve the nearby-donations query while the primary continues to serve writes, fronted by a pooler such as PgBouncer.

- **Sharding the activity and notification collections.** MongoDB's built-in sharding can distribute the activities and notifications collections by user identifier once their size passes a practical single-node threshold.
- **Caching hot queries more broadly.** Beyond the current short-lived Redis cache for search results, category lists and NGO capacity summaries change rarely and can be cached longer with explicit invalidation on write.
- **Event-driven integration.** Replacing some of the direct REST calls between services (for example, Listing to Matching) with a message broker such as Kafka or RabbitMQ would decouple services further and make retries and back-pressure easier to manage at higher volume.
- **Autoscaling on business metrics.** Horizontal Pod Autoscalers currently react to CPU; adding a custom metric such as queue depth or requests per second would react faster to real load.
- **Splitting the Matching Service's workloads.** Embedding generation is CPU-bound while ranking queries are I/O-bound; separating them into two deployments would let each scale according to its own bottleneck.
- **Edge caching for the front end.** Serving the built React application through a content delivery network would reduce latency for users far from the primary region.

These directions were deliberately left as future work so that the current system remains a clear, well-tested reference implementation of the course outcomes, rather than a larger system with untested scale assumptions.

CHAPTER - 12 CONCLUSION

12.1 Summary of the Project

The Food Surplus Redistribution and Community Hunger Reduction Platform was built to close the gap between the large amount of edible food that goes to waste and the communities that need it, by giving donors, NGOs, volunteers and beneficiaries a single, reliable and traceable system. Starting from a clear problem statement, the project moved through requirements, architecture, detailed design, implementation, testing and deployment, and produced a working polyglot backend together with a responsive web front end.

The relational core in PostgreSQL models donors, NGOs, volunteers, beneficiaries, donations, inventory, requests, pickups, distributions and ratings in Boyce-Codd normal form, protected by constraints, indexes, views, stored procedures and triggers, and coordinated through ACID transactions under multi-version concurrency control. MongoDB stores the more flexible user, activity and notification data, and pgvector adds semantic matching and a retrieval-augmented food-safety assistant on top of the same relational engine. Three back-end frameworks, FastAPI, Express and Spring Boot, were used for the parts of the system where each is strongest, joined by an API gateway that centralises authentication, authorisation, rate limiting and error handling. The services are decomposed by business capability, each owns its data, and cross-service consistency for the central reservation workflow is achieved with the Saga pattern and compensating actions rather than a blocking two-phase commit. The system is containerised, deployable to Kubernetes, observable through OpenTelemetry, Prometheus and Grafana, and delivered through an automated pipeline that tests, scans and publishes every change.

Table 12.1 restates, briefly, how the finished system meets each objective set out in Chapter 1.

Table 12.1: Objectives and how they were met

Objective (Chapter 1)	How it was met
Reduce food wastage within the safe-consumption window	Expiry-aware listings, real-time matching and notifications shorten the time between surplus and pickup
Centralised, normalised database with constraints	BCNF schema in PostgreSQL with CHECK constraints enforcing the inventory invariant
Connect all stakeholders through one platform	Role-based accounts for donors, NGOs, volunteers and beneficiaries behind one gateway
Accurate, real-time inventory	Row-level locking and Saga-based reservation prevent double allocation
Efficient pickup and delivery management	Distance- and rating-based volunteer assignment and a tracked pickup workflow
End-to-end traceability	Audit triggers and the donation lifecycle state machine

Objective (Chapter 1)	How it was met
Useful reports with advanced SQL	Joins, CTEs, aggregates and window functions power the analytics dashboard
Right database for each kind of data	PostgreSQL for transactions, MongoDB for documents, pgvector for embeddings, Redis for cache and messaging
Secure, documented REST APIs	JWT, RBAC, rate limiting, OpenAPI in all three frameworks
Independently deployable, resilient microservices	Database-per-service, Saga, timeouts, retries and circuit breaking
Containerised, observable, continuously delivered	Docker, Kubernetes, OpenTelemetry/Prometheus/Grafana, GitHub Actions

12.2 What Was Achieved

The project delivered a functioning platform that a donor, an NGO coordinator, a volunteer and an administrator can use to carry a donation from listing to delivery, with every step recorded and visible. It also delivered, as a direct consequence of the way the system was built, a working demonstration of every course outcome from CO1 to CO6: a normalised relational schema with stored logic, a polyglot data layer including a vector database, three production-style backend frameworks, a decomposed microservice architecture with a documented consistency strategy, and a containerised, observed and automated deployment pipeline. Automated tests, including a concurrency test that proves food is never promised twice, and load tests against defined latency thresholds, give the team confidence that the core guarantees of the system hold under realistic conditions.

12.3 Skills Learned During Development

Working on this project across its six course outcomes gave the team hands-on experience that goes beyond what a single-framework assignment could offer.

- Designing a normalised relational schema from an entity-relationship model and expressing business rules as constraints, views, procedures and triggers rather than only in application code.
- Reasoning about isolation, locking and concurrency, and proving correctness with a concurrency test rather than by inspection alone.
- Choosing between relational, document and vector storage for a given kind of data, and integrating them without sacrificing the consistency that matters most.
- Building REST APIs in three different frameworks and recognising which idioms (dependency injection, middleware, declarative security) recur under different names.

- Designing and implementing a microservice decomposition around business capabilities, and handling distributed failure with sagas and compensations instead of distributed transactions.
- Practising DevOps skills that are often left out of coursework: multi-stage Docker builds, Kubernetes manifests, a CI/CD pipeline with security scanning, and dashboards built from real metrics.
- Working as a small team across different technologies, agreeing on a contract before building against it, and reviewing each other's code in an unfamiliar framework.
- Balancing a social objective, reducing food wastage and hunger, against engineering trade-offs such as consistency versus availability, which made the abstract CAP theorem concrete and memorable.

Overall, the Food Surplus Redistribution and Community Hunger Reduction Platform met its dual purpose: it is a usable tool that could meaningfully help redistribute surplus food in a community, and it is a complete, working record of the database and distributed backend engineering skills built during the course.

CHAPTER - 13 REFERENCES

13.1 Books

1. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
2. Silberschatz, A., Korth, H. F. and Sudarshan, S., *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
3. Kleppmann, M., *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
4. Newman, S., *Building Microservices*, 2nd ed., O'Reilly Media, 2021.
5. Richardson, C., *Microservices Patterns*, Manning Publications, 2018.
6. Date, C. J., *An Introduction to Database Systems*, 8th ed., Pearson, 2004.
7. Connolly, T. and Begg, C., *Database Systems: A Practical Approach to Design, Implementation and Management*, 6th ed., Pearson, 2014.
8. Fielding, R. T., *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. dissertation, University of California, Irvine, 2000.
9. Fowler, M., *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

13.2 Research Papers and Technical Reports

10. Chen, P. P., "The Entity-Relationship Model: Toward a Unified View of Data," *ACM Transactions on Database Systems*, vol. 1, no. 1, 1976.
11. Garcia-Molina, H. and Salem, K., "Sagas," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1987.
12. Gilbert, S. and Lynch, N., "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News*, vol. 33, no. 2, 2002.
13. Malkov, Y. A. and Yashunin, D. A., "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, 2020.
14. Lewis, P. et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
15. Reimers, N. and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.
16. United Nations Environment Programme, *Food Waste Index Report 2024*, UNEP, Nairobi, 2024.
17. Food Safety and Standards Authority of India, *Guidance Note on Surplus Food Recovery and Distribution*, FSSAI, New Delhi, 2021.

18. Brown, S., *The C4 Model for Visualising Software Architecture*, self-published technical report, 2018. Available: <https://c4model.com>

13.3 Official Documentation

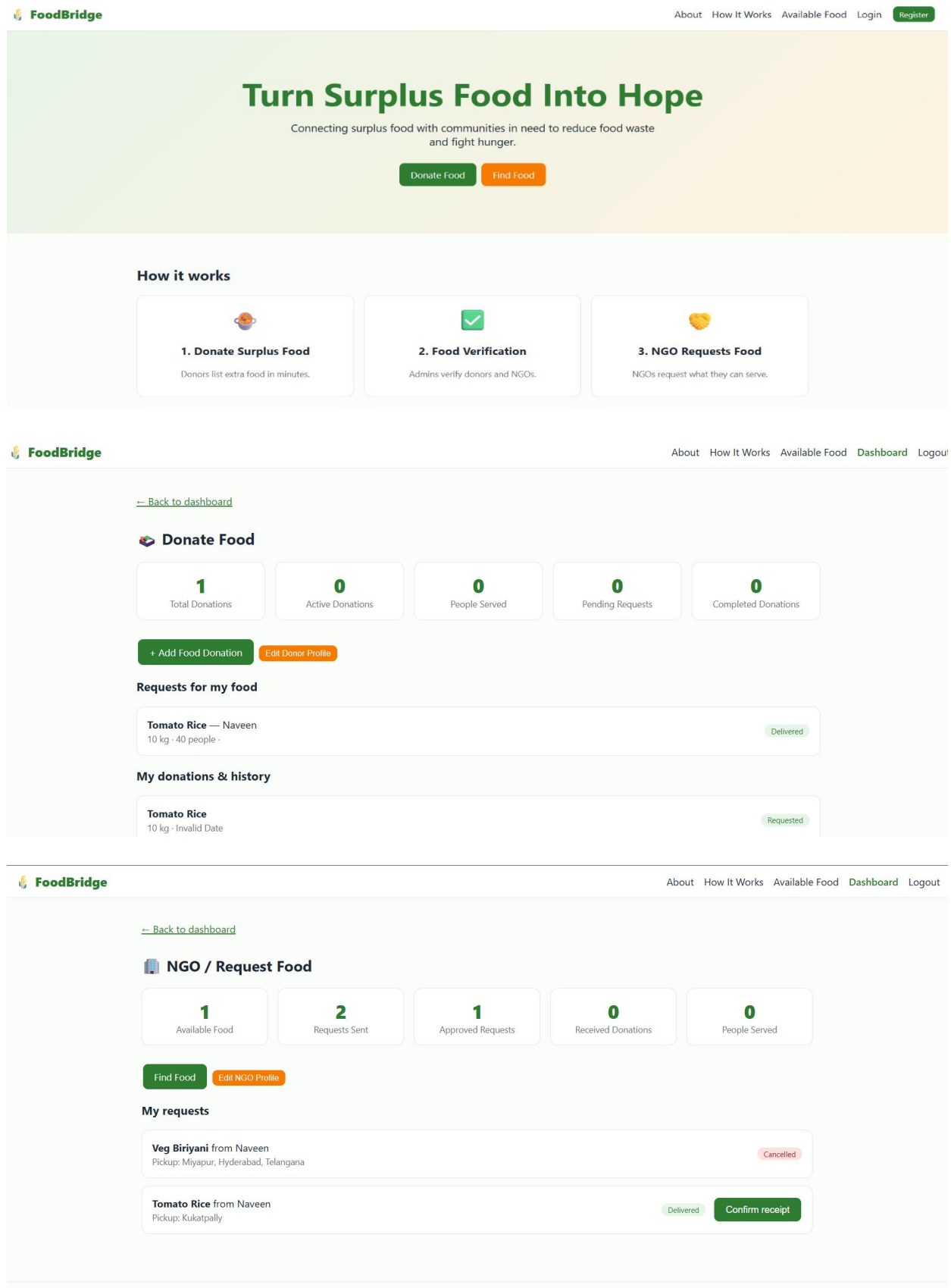
19. FastAPI, "FastAPI Documentation." [Online]. Available: <https://fastapi.tiangolo.com>
20. Pydantic, "Pydantic Documentation." [Online]. Available: <https://docs.pydantic.dev>
21. SQLAlchemy, "SQLAlchemy 2.0 Documentation." [Online]. Available: <https://docs.sqlalchemy.org>
22. PostgreSQL Global Development Group, "PostgreSQL 16 Documentation." [Online]. Available: <https://www.postgresql.org/docs/16/>
23. pgvector, "pgvector: Open-source vector similarity search for Postgres." [Online]. Available: <https://github.com/pgvector/pgvector>
24. MongoDB Inc., "MongoDB Manual." [Online]. Available: <https://www.mongodb.com/docs/manual/>
25. OpenJS Foundation, "Node.js Documentation." [Online]. Available: <https://nodejs.org/en/docs>
26. Mongoose, "Mongoose Documentation." [Online]. Available: <https://mongoosejs.com/docs/>
27. Socket.io, "Socket.IO Documentation." [Online]. Available: <https://socket.io/docs/>
28. VMware / Spring, "Spring Boot Reference Documentation." [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
29. VMware / Spring, "Spring Security Reference." [Online]. Available: <https://docs.spring.io/spring-security/reference/>
30. Oracle, "Java Persistence API (JPA) Specification." [Online]. Available: <https://jakarta.ee/specifications/persistence/>
31. Redis Ltd., "Redis Documentation." [Online]. Available: <https://redis.io/docs/>
32. OpenTelemetry, "OpenTelemetry Documentation." [Online]. Available: <https://opentelemetry.io/docs/>
33. Prometheus Authors, "Prometheus Documentation." [Online]. Available: <https://prometheus.io/docs/>
34. Grafana Labs, "Grafana Documentation." [Online]. Available: <https://grafana.com/docs/>
35. OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: <https://owasp.org/Top10/>
36. IETF, "RFC 7519: JSON Web Token (JWT)," 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>

37. IETF, "RFC 6749: The OAuth 2.0 Authorization Framework," 2012. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6749>
38. OpenAPI Initiative, "OpenAPI Specification v3.1." [Online]. Available: <https://spec.openapis.org/oas/v3.1.0>
39. Docker Inc., "Docker Documentation." [Online]. Available: <https://docs.docker.com>
40. pytest, "pytest Documentation." [Online]. Available: <https://docs.pytest.org>
41. OpenJS Foundation, "Jest Documentation." [Online]. Available: <https://jestjs.io/docs/getting-started>
42. OpenStreetMap Foundation, "Nominatim Usage Policy." [Online]. Available: <https://operations.osmfoundation.org/policies/nominatim/>
43. Flask, "Flask Documentation," and Django Software Foundation, "Django REST Framework Documentation." [Online]. Available: <https://flask.palletsprojects.com> and <https://www.django-rest-framework.org>
44. Pinecone Systems and Weaviate B.V., product documentation. [Online]. Available: <https://docs.pinecone.io> and <https://weaviate.io/developers/weaviate>
45. Grafana Labs, "k6 Documentation." [Online]. Available: <https://k6.io/docs/>
46. The Kubernetes Authors, "Kubernetes Documentation." [Online]. Available: <https://kubernetes.io/docs/home/>
47. Postman Inc., "Postman Learning Center." [Online]. Available: <https://learning.postman.com>

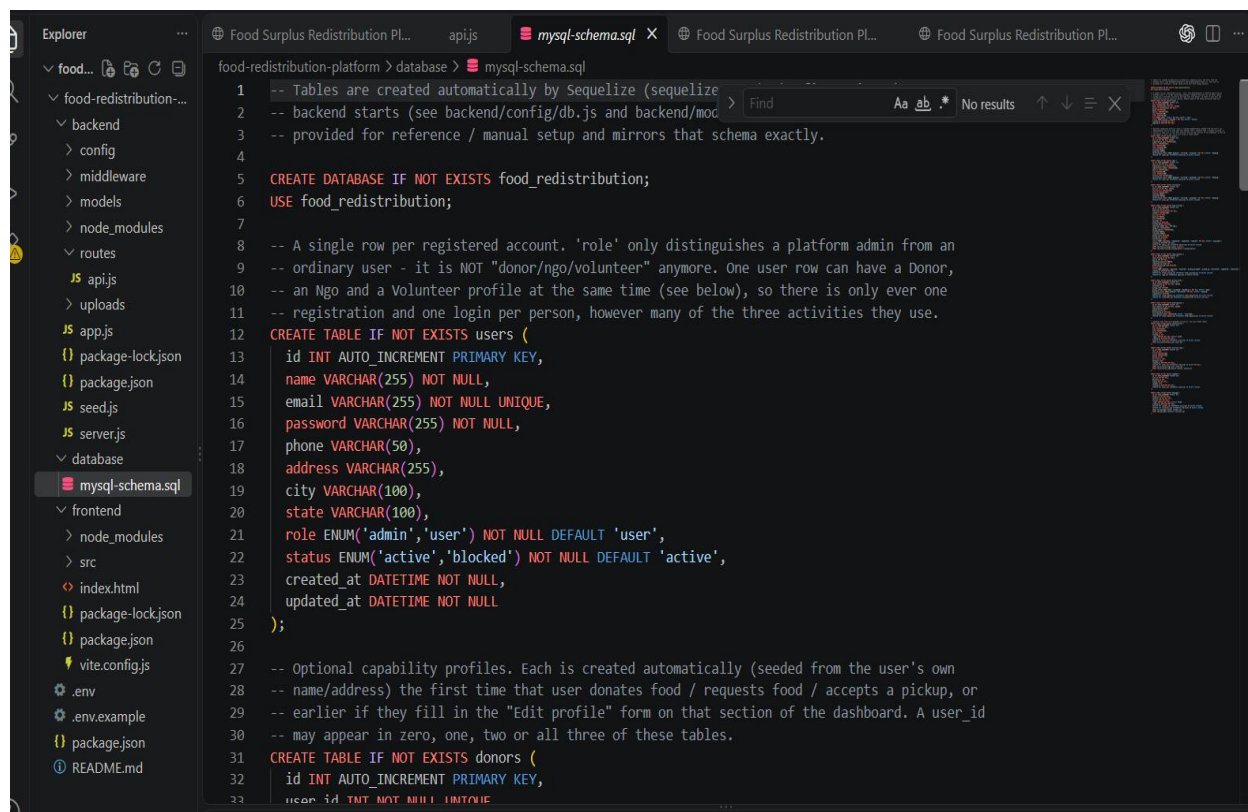
Note: All web resources listed above were last accessed in September 2026. Where a specific version is relevant to this project, the version is stated in Chapter 3 (Technology Stack) rather than repeated here.

CHAPTER - 14 APPENDICES

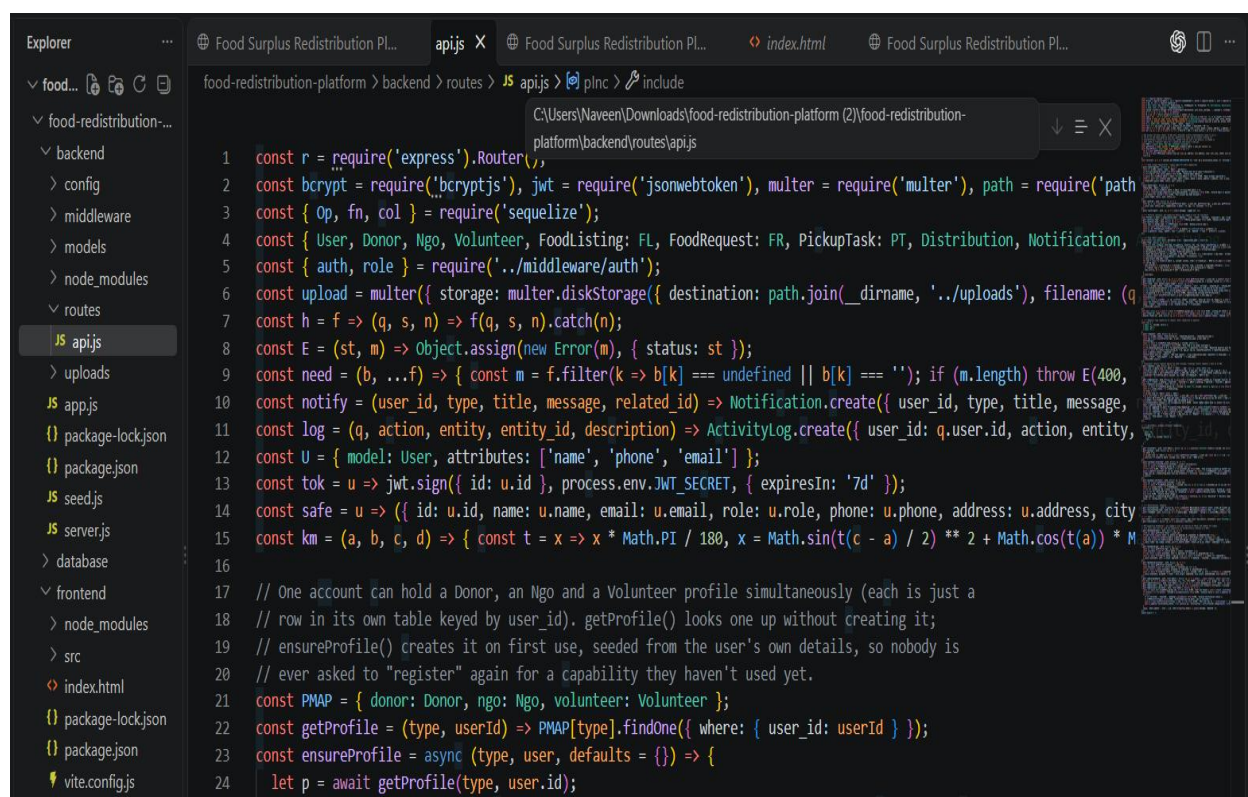
Screenshot of the app:



Sample code Snippets:



```
1  -- Tables are created automatically by Sequelize (sequelize
2  -- backend starts (see backend/config/db.js and backend/mod
3  -- provided for reference / manual setup and mirrors that schema exactly.
4
5  CREATE DATABASE IF NOT EXISTS food_redistribution;
6  USE food_redistribution;
7
8  -- A single row per registered account. 'role' only distinguishes a platform admin from an
9  -- ordinary user - it is NOT "donor/ngo/volunteer" anymore. One user row can have a Donor,
10 -- an Ngo and a Volunteer profile at the same time (see below), so there is only ever one
11 -- registration and one login per person, however many of the three activities they use.
12 CREATE TABLE IF NOT EXISTS users (
13   id INT AUTO_INCREMENT PRIMARY KEY,
14   name VARCHAR(255) NOT NULL,
15   email VARCHAR(255) NOT NULL UNIQUE,
16   password VARCHAR(255) NOT NULL,
17   phone VARCHAR(50),
18   address VARCHAR(255),
19   city VARCHAR(100),
20   state VARCHAR(100),
21   role ENUM('admin','user') NOT NULL DEFAULT 'user',
22   status ENUM('active','blocked') NOT NULL DEFAULT 'active',
23   created_at DATETIME NOT NULL,
24   updated_at DATETIME NOT NULL
25 );
26
27 -- Optional capability profiles. Each is created automatically (seeded from the user's own
28 -- name/address) the first time that user donates food / requests food / accepts a pickup, or
29 -- earlier if they fill in the "Edit profile" form on that section of the dashboard. A user_id
30 -- may appear in zero, one, two or all three of these tables.
31 CREATE TABLE IF NOT EXISTS donors (
32   id INT AUTO_INCREMENT PRIMARY KEY,
33   user_id INT NOT NULL UNIQUE
```



```
1  const r = require('express').Router();
2  const bcrypt = require('bcryptjs'), jwt = require('jsonwebtoken'), multer = require('multer'), path = require('path')
3  const { Op, fn, col } = require('sequelize');
4  const { User, Donor, Ngo, Volunteer, FoodListing: FL, FoodRequest: FR, PickupTask: PT, Distribution, Notification,
5  const { auth, role } = require('../middleware/auth');
6  const upload = multer({ storage: multer.diskStorage({ destination: path.join(__dirname, '..', 'uploads'), filename: (q
7  const h = f => (q, s, n) => f(q, s, n).catch(n);
8  const E = (st, m) => Object.assign(new Error(m), { status: st });
9  const need = (b, ...f) => { const m = f.filter(k => b[k] === undefined || b[k] === ''); if (m.length) throw E(400,
10 const notify = (user_id, type, title, message, related_id) => Notification.create({ user_id: user_id, type, title, message,
11 const log = (q, action, entity, entity_id, description) => ActivityLog.create({ user_id: q.user_id, action, entity,
12 const U = { model: User, attributes: ['name', 'phone', 'email'] };
13 const tok = u => jwt.sign({ id: u.id }, process.env.JWT_SECRET, { expiresIn: '7d' });
14 const safe = u => ({ id: u.id, name: u.name, email: u.email, role: u.role, phone: u.phone, address: u.address, city
15 const km = (a, b, c, d) => { const t = x => x * Math.PI / 180, x = Math.sin(t(c - a) / 2) ** 2 + Math.cos(t(a)) * M
16
17 // One account can hold a Donor, an Ngo and a Volunteer profile simultaneously (each is just a
18 // row in its own table keyed by user_id). getProfile() looks one up without creating it;
19 // ensureProfile() creates it on first use, seeded from the user's own details, so nobody is
20 // ever asked to "register" again for a capability they haven't used yet.
21 const PMAP = { donor: Donor, ngo: Ngo, volunteer: Volunteer };
22 const getProfile = (type, userId) => PMAP[type].findOne({ where: { user_id: userId } });
23 const ensureProfile = async (type, user, defaults = {}) => {
24   let p = await getProfile(type, user.id);
25   if (!p) {
```

Installation/setup instructions

The Food Surplus Redistribution and Community Hunger Reduction Platform is designed as a web-based application that allows donors to offer surplus food and enables recipients or volunteers to manage and collect available food donations. The project consists of a frontend interface and a backend server connected to a database.

First, download or copy the complete project folder to your local computer. If the project is shared as a ZIP file, extract it using any standard extraction tool. Make sure that all project files and folders, including the frontend, backend, configuration files, and package files, are present.

Next, open the project folder in Visual Studio Code. Open the integrated terminal and navigate to the backend directory. Install the required dependencies using the package manager specified in the project. The backend is then started using the project's start command.

Before running the application, make sure that the required database service is available and that the database configuration in the backend is correctly set. The database stores important information such as user accounts, food donations, pickup details, and donation requests.

After starting the backend server, open the frontend application in the browser using the local server address provided by the project. The application can then be accessed through modern browsers such as Google Chrome, Microsoft Edge, or Firefox.

Once the application is running, users can register and log in to access the available features. The platform provides different functions for donors, recipients, and administrators according to their roles.

The project is intended for educational demonstration and can be executed locally using Visual Studio Code, Node.js, and the configured database environment.

Geo Tag photos with guide



